TECHNICAL REPORT

JewelRoute
Frontend ↔ Backend Integration

How the Next.js web app and the NestJS API connect: every endpoint and what it does, the API design, the data flow behind each click, and flowcharts of the key processes.

Audience	Founder and CTO reviewing the system
Date	21 October 2026 (times in IST, UTC+5:30)
Source	Repository `jewelroute`, branch head `831d3d2` (clean working tree when this was generated). Revised for **v3**: credit-aware orders (CONFIRMED or PENDING_APPROVAL with reasons CREDIT_LIMIT / ORDER_THRESHOLD; approve, reject, settle, cancel), receivables and the shared credit rule, tenant approval threshold, rejection un-selling memo conversions, the memo-sale preview, one activity model with indexed refs and cursor paging, one global lock order, and the P7 walkthrough fixes (decision notes and 10 s pending polling in the field). This report documents the code on disk, not plans.
Apps	`apps/web` Next.js 15 (App Router, React, TanStack Query) · `apps/api` NestJS 11 + Prisma + PostgreSQL · `packages/shared` pricing, geo, dates, event types
Method	Every controller, service, migration and web call site was read (v3 delta: `docs/CONTRACT.md` changelog and § v3, `core/credit.ts`, `core/activity.ts`, `core/event-refs.ts`, migrations `…0300`–`…0800`, `lib/sync.ts`). Endpoint list generated from `@Controller`/`@Get|@Post|@Patch|@Delete` decorators; click mappings traced from `api(`, `useMutation`, `useAction`, `useWrite` and `useSync` call sites.

Contents

1 **Summary** 4

1.1 Two apps, one contract 4
1.2 How they talk 4
1.3 What a reviewer should know 5
1.4 Pages and record tabs 6

2 **Architecture and connection** 7

2.1 The request path 8
2.2 Authentication and session refresh 8
2.3 NestJS pipeline: guards, interceptors, filter, modules 8
2.4 Envelope, error codes and request ids 9
2.5 Idempotency keys 9
2.6 Tenant scoping and RBAC 10
2.7 Uploads: Vercel Blob or direct 10
2.8 Outbound webhooks 10
2.9 Public acknowledgement page 10
2.10 Credit-aware orders and the global lock order (v3) 11
2.11 One activity model (v3) 11

3 **API design** 12

3.1 Principles 12
3.2 Versioning: v1 and additive v2 12
3.3 Conventions at a glance 13
3.4 Error catalogue 13

4 **Complete endpoint reference** 15

4.1 Auth 15
4.2 Field: today, visits, check-in, proofs, notes, catalog 15
4.3 Field: uploads and files 17
4.4 Field: orders 17
4.5 Field: memos and retailer acknowledgement 17
4.6 Admin: home, dashboard, activity, search, settings 19
4.7 Admin: shops 19
4.8 Admin: visits and assignments 20
4.9 Admin: agents 21
4.10 Admin: orders 21
4.11 Admin: memos 22
4.12 Admin: designs, pieces and custody 22
4.13 Admin: gold rates and audit 23
4.14 Webhooks and integrations 24
4.15 Public 25
4.16 Ops 25

5 **Data model** 26

5.1 Tables by area 26
5.2 Database-enforced invariants 27

5.3 Relationship diagram 29

6 Click-by-click data flow 30

6.1 Login 30
6.2 Field: Today and opening a visit 30
6.3 Field: geofence and check-in 31
6.4 Field: arrival photo (proof) 32
6.5 Field: visit workspace, notes, catalog, order 32
6.6 Field: memo, retailer acknowledgement, return, complete 34
6.7 Admin: Home, search, visits and routes 35
6.8 Admin: orders, memos, samples, catalog, gold rate 37
6.9 Admin: shops and assigned agents 38
6.10 Admin: webhooks and integrations 39
6.11 Flow: Rahul books a big order → pending approval → Priya decides → Rahul sees why 39
6.12 Flow: sell from memo with preview → approval → reject returns the pieces to the memo 40
6.13 Flow: one event reaches every activity feed 41

7 Flowcharts 42

7.1 Request lifecycle 42
7.2 Field visit flow 44
7.3 Custody state machine for a sample piece 46
7.4 Memo state machine 47
7.5 Order state machine 48
7.6 Visit state machine 48
7.7 Webhook delivery pipeline 49
7.8 Sequence: Rahul leaves AJ-S-000123 on memo at Sharma Jewellers 50
7.9 Sequence: a big order waits for approval, Priya decides, Rahul sees why 51
7.10 Sequence: sell from memo with preview, then the sale is rejected and un-sold 52
7.11 Activity propagation: one event, every feed 53

8 Sync and consistency 54

8.1 The backend is the source of truth 54
8.2 Race and duplicate protections 54
8.3 Failure handling 55

9 Appendix 57

9.1 Environment variables 57
9.2 Running locally 57
9.3 Glossary 57

1 Summary

JewelRoute is a field-sales system for jewellery makers who sell through retailers. A manager plans routes; field agents visit shops, check in with server-verified GPS, attach a photo, book orders priced from the live gold rate, and leave physical samples on *memo* (consignment) with a retailer who acknowledges receipt from their own phone. Since v3 an order is **confirmed straight away** unless a server-side rule trips (the shop would exceed its credit limit, or the order reaches the tenant's approval threshold); only then does it wait for a manager. Every custody move of every sample is recorded in an append-only ledger, and state changes are published to customer systems as signed webhooks.

1.1 Two apps, one contract

- **Web** (`apps/web`, Next.js 15): three surfaces in one app. The field app at `/field/*` is mobile-first (Today, Visits, Catalog, Profile tabs). The admin console is at `/admin/*` (Home, Visits, Agents, Shops, Orders, Memos, Samples, Catalog, Integrations, Settings). The public retailer acknowledgement page is at `/ack/[token]`. There are also a marketing page at `/` and `/developers` docs. 26 pages in all (§1.4).
- **API** (`apps/api`, NestJS 11 + Prisma): all business rules, pricing, custody, idempotency, tenancy and webhooks. PostgreSQL is the single source of truth. The web never patches derived state locally: it writes, then refetches.
- **Shared** (`packages/shared`): pricing maths (`priceDesign`, `orderTotals`), the geofence decision, IST business dates, money formatting and the webhook event list. The API uses them as the authority and the web uses the same code for display only.

1.2 How they talk

Browser → same-origin `/api/v1/…` → Next.js rewrite in `apps/web/next.config.ts` (`/api/:path*` → `${API_ORIGIN}/:path*`) → NestJS `/v1/*` (throttler → auth/RBAC guard → zod validation → service in a Prisma transaction) → PostgreSQL. Responses come back in one envelope, `{ok:true,data}` or `{ok:false,error:{code,message,details},requestId}`. Auth is two httpOnly first-party cookies (`jr_at` access JWT, 15 min; `jr_rt` rotating refresh token, 30 days), so the browser sends them with no CORS. State changes also write an `OutboxEvent` row in the same transaction. After the response, a post-commit interceptor kicks the dispatcher, which fans events out to subscribed endpoints as HMAC-signed HTTP POSTs with a retry ladder.

89 HTTP routes: 4 auth · 25 field · 41 admin · 12 webhook admin · 2 public · 5 ops (82 before v3; +7)	**25** tables (Prisma models), 9 migrations; 3 append-only by trigger, 2 deferred sale guards, 2 partial unique indexes, 19 CHECKs	**17 + 1** webhook event types (5 new in v3), plus `ping` for test sends	**50 + 14** API integration tests on real Postgres (12 + 16 + 4 + 14 + 4) + 14 shared unit tests; 31-check HTTP e2e smoke

1.3 What a reviewer should know

Topic	Finding (from the code on disk)
Strong guarantees	Money and custody creates (`POST /orders`, `/memos`, `/memos/:id/return`, `/memos/:id/convert`) **require** an `Idempotency-Key`; the key row is inserted inside the business transaction, so it commits or rolls back with the effect. Concurrency is guarded by row locks (`FOR UPDATE`) taken in **one global order** (Memo → Visit → SamplePiece by id → Shop → Order, §2.10), guarded updates (… `WHERE status='PENDING_APPROVAL'`), and database invariants (one open visit per agent, one OUT memo line per piece, append-only ledgers, deferred "a SOLD piece has a live order" triggers).
v3: approval is a rule	The credit rule runs inside the order transaction with the shop row locked (`CreditService.position(…, {lock:true})`): `CREDIT_LIMIT` when projected exposure > limit, `ORDER_THRESHOLD` when the total ≥ the tenant threshold. No rule → `CONFIRMED` (counts as outstanding at once); otherwise `PENDING_APPROVAL` with the reasons and a frozen `CreditPosition`. The field order review and the memo-sale preview show the same numbers beforehand (`GET /visits/:id/credit`, `GET /memos/:id/convert-preview`); the server decides again when it writes.
v3: reject un-sells	Rejecting an order converted from a memo moves its pieces SOLD → the memo's shop (custody reason `SALE_REJECTED`), puts their memo items back to OUT and recomputes the memo status, in the same transaction. Deferred constraint triggers refuse to commit any rejection or cancel that would leave a piece SOLD without a live order. A converted order cannot be cancelled (`409 ORDER_NOT_CANCELLABLE`).
v3: one activity model	Every `OutboxEvent` now stores indexed refs (`shopId, agentId, visitId, orderId, memoId, pieceIds[]`, back-filled for old rows). `ActivityService` is the only projection to `ActivityItem {kind, refs, detail}`; Home and all six record types render it through one component, `components/app/activity-feed.tsx`, calling `GET /admin/activity?<ref>=…&limit=…&before=…` with cursor paging. The old "useActivity is unused" gap is closed.
Routes with no web caller	v1 reads kept for API clients and `/developers`: `GET /admin/dashboard`, `/admin/pieces/:id/history`, `/admin/audit`, `/orders/:id`, `/memos/:id`; `GET /admin/shops/:id/credit` (the shop record reads `ShopRow.credit` instead); and the ops routes. Every other route has a caller (verified by matching each path against `apps/web/src`).
One refresh mechanism	Every write in the web app refreshes through `useSync()` and the single `AFFECTS` map in `lib/sync.ts`, now 22 change kinds. v3 added `order.settled`, `order.cancelled`, `order.unsold` (a rejected memo sale back on the memo), `settings.saved`, and the `credit` / `convert-preview` keys to every change that moves exposure.
Retry cadence depends on traffic	The only scheduled trigger is a **daily** Vercel cron (`0 3 * * *` UTC = 08:30 IST) on `/v1/internal/dispatch`. Between runs, due retries are attempted only when a write kicks the dispatcher or an admin opens Home, Dashboard or Deliveries (at most once per 10 s per instance). A FAILED delivery's 30 s back-off is therefore a minimum, not a schedule.
Optional keys	Check-in, proof attach and complete accept an optional key. The web sends one for check-in and proofs, but not for complete or for the field memo extend. Admin writes (approve, reject, settle, cancel, settings) carry no key; they rely on guarded updates (`ORDER_ALREADY_DECIDED`, `ORDER_ALREADY_SETTLED`) and unique constraints, except piece intake (`POST /admin/designs/:id/pieces`), where a double submit would create two batches. The UI disables the button while pending.
Small inconsistencies	The shop form accepts geofence radius 20–1000 m while the API accepts 10–2000 m. `JWT_REFRESH_SECRET` is validated at boot but unused, because refresh tokens are opaque random strings stored as SHA-256. Activity kind `rate` is reserved: setting a gold rate still emits no event.
Not verified here	The test suites and e2e smoke were not executed for this report. Counts and behaviours come from reading the source (tests counted from `it(` calls).

1.4 Pages and record tabs

26 routable pages (`apps/web/src/app/**/page.tsx`; `/field` only redirects to `/field/today` and is not counted). Record pages open with a property panel on the left and the tabs below; list rows also open a side preview (`components/admin-v2/previews.tsx`: order, shop, memo, sample, visit, agent).

Page	Tabs / modes	Main API reads (polling)
Public and auth (4)		
`/` · `/developers`	marketing; API docs	none (static content)
`/login`	form + two demo buttons	`POST /auth/login`
`/ack/[token]`	retailer receipt	`GET /public/memo-ack/:token` (re-read every 5 s while pending)
Field app (6) — bottom tabs Today · Visits · Catalog · Profile		
`/field/today`	route list + map	`GET /field/today` (60 s)
`/field/visits`	Recent · Upcoming	`GET /field/visits?scope=`
`/field/visits/[id]`	stages Check in → Arrival photo → Sell and complete	`GET /visits/:id` (10 s while an order awaits a decision; 3 s on the ack screen), `POST …/geofence-check` (5 s)
`/field/visits/[id]/sale`	Order · Memo; catalog → review	`GET /visits/:id/catalog`, `GET /visits/:id/credit?amountPaise=`
`/field/catalog` · `/field/profile`	browse · bag, memos, week	`GET /field/catalog` · `GET /field/profile`
Admin console (16) — sidebar Home · Field (Visits, Agents, Shops) · Sales (Orders, Memos, Samples) · Workspace (Catalog, Integrations, Settings)		
`/admin` Home	attention · map · activity (agent filter)	`GET /admin/home` (10 s), `GET /admin/activity?limit=20` (10 s)
`/admin/visits` · `[id]`	Today · Yesterday · Last 7 days · record: **Overview · Activity · Audit**	`GET /admin/visits`, `/admin/visits/:id` (15 s), activity `?visitId=`
`/admin/agents` · `[id]`	record: **Today's route · Bag · Memos · Orders · Activity**	`GET /admin/agents`, `/admin/agents/:id` (15 s), activity `?agentId=`
`/admin/shops` · `[id]`	record: **Overview · Visits · Orders · Memos · Activity** (credit and exposure on Overview)	`GET /admin/shops` (30 s), `/admin/shops/:id` (15 s), activity `?shopId=`
`/admin/orders` · `[id]`	All · Pending approval · Confirmed · Approved · Unsettled · Rejected · Cancelled · record: **Overview · History · Activity**	`GET /admin/orders`, `/admin/orders/:id` (15 s), activity `?orderId=`
`/admin/memos` · `[id]`	All active · Due today · Overdue · Returned · record: **Overview · History · Activity**	`GET /admin/memos`, `/admin/memos/:id` (20 s), activity `?memoId=`
`/admin/samples` · `[id]`	record: **Custody · Activity**	`GET /admin/pieces`, `/admin/pieces/:id` (20 s), activity `?pieceId=`
`/admin/catalog`	design grid + sheet	`GET /admin/designs`
`/admin/integrations`	endpoint panel: **Deliveries · Test receiver · Verify signatures**	`GET /admin/webhooks`, `…/deliveries` (cursor, 5 s), `…/received` (5 s)
`/admin/settings`	**Gold rate · Approvals · Workspace**	`GET /admin/gold-rates`, `GET /admin/settings`

2 Architecture and connection

One browser-visible origin (the Next.js app) fronts the API. Cookies stay first-party, the browser never needs CORS, and the API remains independently deployable (on Vercel, `apps/api/vercel.json` routes every path to one serverless function that boots Nest once per instance, see `src/serverless.ts`).

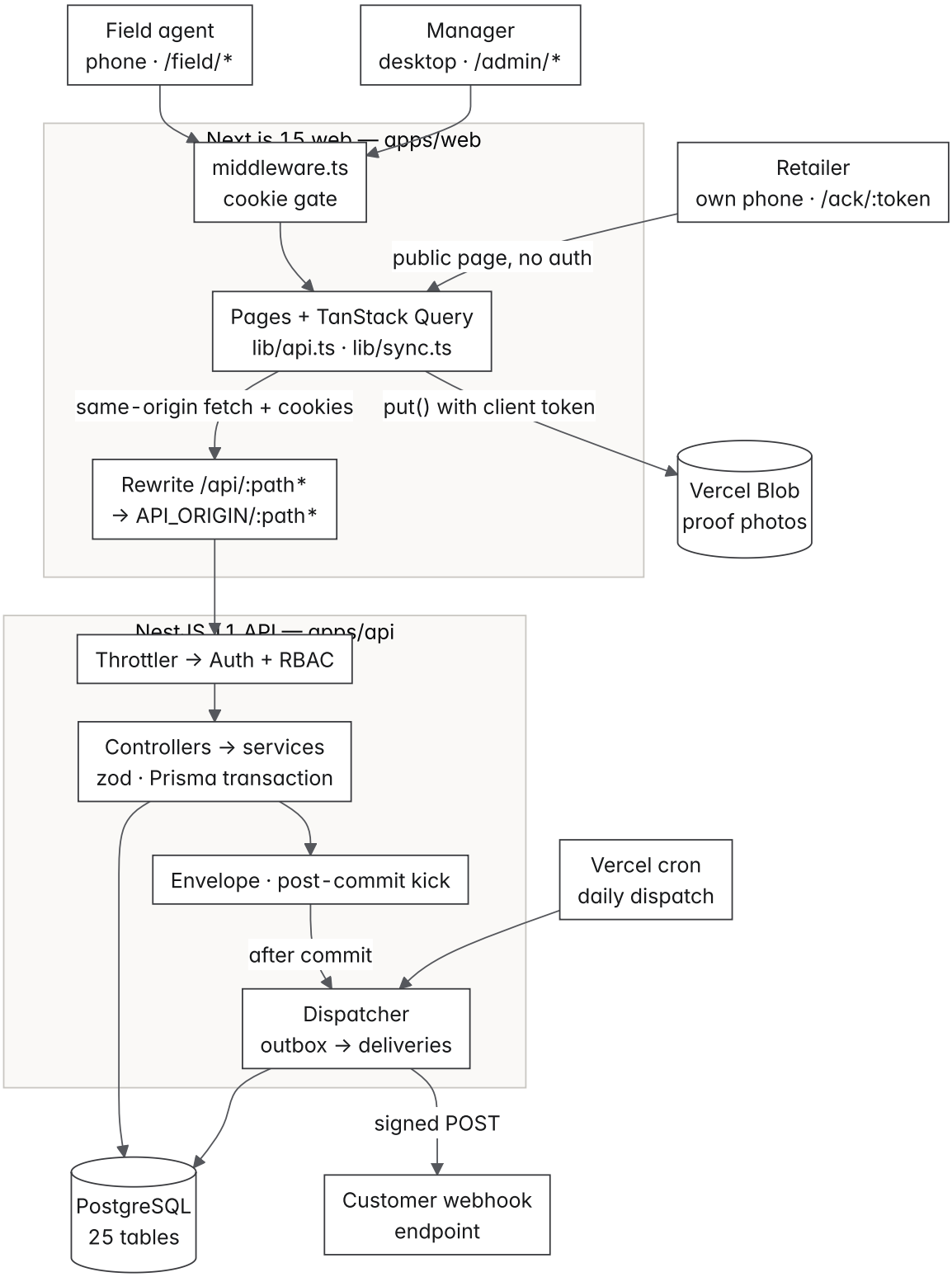


Figure 1 — System context. All browser traffic, including the public acknowledgement page, enters through the Next.js origin and is proxied to the API. Proof photos go straight from the phone to Vercel Blob when a Blob token is configured; otherwise they go through the API (direct mode, stored in Postgres).

2.1 The request path

1. **Browser**: every authenticated call goes through `api()` in `apps/web/src/lib/api.ts`. It prefixes `/api/v1`, sets `credentials:"include"` and `cache:"no-store"`, JSON-encodes the body (or passes `FormData`), and adds `Idempotency-Key` when asked (`true` → fresh `crypto.randomUUID()`, or a caller-supplied stable key).
2. **Next.js**: `next.config.ts` rewrites `/api/:path*` to `${API_ORIGIN}/:path*` (default `http://localhost:4000`) and adds security headers to every page (`X-Content-Type-Options`, `Referrer-Policy`, `X-Frame-Options: SAMEORIGIN`, `Permissions-Policy: camera=(self), geolocation=(self)`).
3. **Middleware** (`src/middleware.ts`, matcher `/field/:path*` and `/admin/:path*`): a cheap gate that redirects to `/login?next=…` when neither `jr_at` nor `jr_rt` is present. It does no verification; real authorization happens in the API on every call. After login, `login-form.tsx` sends ADMIN users to `/admin` and FIELD_AGENT users to `/field/today`, and honours `?next=` only inside the role's own area.
4. **NestJS** (`src/app.ts`): helmet, cookie-parser, 1 MB JSON limit, CORS allowlist (a fallback, since the proxy makes calls same-origin), pino request logging. Then the global guards, handler, interceptors and exception filter described below.

2.2 Authentication and session refresh

Login	`POST /v1/auth/login` looks the user up by email (optionally scoped by `tenantSlug`; an email that exists in two tenants without a slug fails), compares with bcrypt, and compares against a dummy hash when the user is missing so timing does not reveal existence. It issues an access JWT and a refresh token, sets both cookies, and also returns them in the body for API clients and the e2e script.
Access token (`jr_at`)	JWT signed with `JWT_ACCESS_SECRET`, issuer `jewelroute`, 15 min. Claims: `sub` (user id), `tid` (tenant id), `tslug`, `role`, `name`. Cookie flags: httpOnly, `SameSite=Lax`, `Secure` in production, path `/`. `AuthGuard` also accepts `Authorization: Bearer`. An expired token gives `401 TOKEN_EXPIRED`; a bad one gives `401 INVALID_TOKEN`.
Refresh token (`jr_rt`)	Opaque `rt_<32 random bytes>`, stored only as SHA-256 in `RefreshToken`, 30 days. `/auth/refresh` claims the row with a conditional `updateMany … revokedAt:null`, so a token can be used once. Presenting an already-revoked token revokes every live token of that user (theft signal).
Client refresh on 401	In `api()`, a 401 on any non-`/auth/*` path triggers a single-flight `POST /api/v1/auth/refresh` (concurrent 401s share one promise) and one retry of the original call. If the refresh fails, the browser goes to `/login?next=<current path>`. The XHR upload path (`proof-upload.ts`) calls `api("/auth/me")` on 401, which runs the same refresh, then retries the upload once.
Logout	`POST /v1/auth/logout` revokes the presented refresh token and clears both cookies. Deactivating an agent (`PATCH /admin/agents/:id {active:false}`) revokes all their refresh tokens, but an issued access JWT stays valid for up to 15 min (documented known gap).

2.3 NestJS pipeline: guards, interceptors, filter, modules

Stage	Implementation (file)
Throttler guard	`AppThrottlerGuard` (`app.ts`): 1200 requests/min per client IP (first `X-Forwarded-For` hop). Login is tracked per IP plus email. Route overrides: login 10/min, refresh 30/min, ack verify 20/min, public ack view 30/min, public confirm 10/min, demo receiver 120/min; health and internal routes skip throttling. Rejections give `429 RATE_LIMITED`.
Auth + RBAC guard	`AuthGuard` (`common/auth.ts`): `@Public()` routes pass. Otherwise it verifies the JWT and checks `@Requires(...)` permissions against `ROLE_PERMISSIONS`: `FIELD_AGENT → [field]`, `ADMIN → [admin, admin.read, field]`. A missing permission gives `403 FORBIDDEN`.

Stage	Implementation (file)
Validation	Every body and query is parsed with zod via `parse()` (`common/http.ts`). A failure gives `400 VALIDATION_FAILED` with `details:[{path,message}]`.
Request context	`@Ctx()` builds `ReqCtx`: user (from the verified JWT), requestId, IP, `Idempotency-Key` (trimmed, ≤200 chars), method, path and body. **tenantId always comes from the JWT**, never from the request.
Envelope interceptor	`EnvelopeInterceptor` wraps every successful return as `{ok:true,data}`. `installBigIntJson()` serialises BigInt (paise) as a JSON number.
Post-commit dispatch	`PostCommitDispatchInterceptor`: after any successful non-GET/HEAD/OPTIONS request (except `/auth/`, `/internal/dispatch` and `/dev/webhook-receiver`) it calls `Dispatcher.kick()`. Services commit their transaction before returning, so the outbox rows are already visible. On Vercel, `waitUntil` keeps the function alive until delivery finishes.
Exception filter	`AllExceptionsFilter`: maps `AppError`, Nest `HttpException`, throttler, zod, Prisma `P2025` (→404), serialization or deadlock failures (→`409 CONCURRENT_UPDATE`), body too large (413) and malformed JSON (400) to `{ok:false,error,requestId}`. Errors of 500 and above are logged.
Modules	`CoreModule` (global: Rates, Outbox, Audit, Idempotency, Views, Custody, Dispatcher), `PrismaModule`, `AuthModule`, `VisitsModule`, `UploadsModule`, `OrdersModule`, `MemosModule`, `AdminModule` (Admin + Records controllers), `WebhooksModule` (Webhooks + Ops controllers). Swagger UI is at `/docs` and JSON at `/docs-json`.

2.4 Envelope, error codes and request ids

- **Success**: `{"ok":true,"data":…}`. `api()` returns `json.data`.
- **Error**: `{"ok":false,"error":{"code":"OUTSIDE_GEOFENCE","message":"…","details":{…}},"requestId":"req_01J…"}`. `api()` throws `ApiError(status, code, message, details, requestId)`, and screens branch on `code` (catalogue in §3.4).
- **Request ids**: a client-supplied `X-Request-Id` matching `^[A-Za-z0-9._-]{6,100}$` is kept; otherwise the API mints `req_<ULID>`. The id is echoed in the `x-request-id` response header (exposed via CORS), included in every error body, attached to log lines, and stored on `AuditLog.requestId`.
- **Retries in the browser**: the TanStack default (`components/providers.tsx`) never retries a 4xx and retries 5xx or network errors up to twice. Default `staleTime` is 10 s, with `refetchOnWindowFocus` on.

2.5 Idempotency keys

`IdempotencyService.run()` (`core/idempotency.ts`) inside one Prisma transaction:

1. Insert `(tenantId, key, userId, fingerprint)` into `IdempotencyKey` with `ON CONFLICT DO NOTHING`. The fingerprint is SHA-256 of `method + path + canonical JSON body`. A concurrent duplicate blocks on the unique slot until the first transaction ends.
2. The winner runs the operation and stores the response body in the same transaction, so the stored response exists only if the effect committed. Errors roll the key back too, so a retry after a failure re-executes (test F6).
3. The loser reads the committed row. Same fingerprint and user returns the stored response. A different body gives `422 IDEMPOTENCY_KEY_REUSED`. A row with no response yet gives `409 IDEMPOTENCY_IN_PROGRESS`.

Second line of defence: orders and memos store the key as `clientOpId` (unique per tenant). On a unique violation the service returns the existing record. **Client side**: `opKey.for(scope, signature)` in `components/field/guard.ts` persists `{sig, key}` in sessionStorage per action scope (`checkin:<visit>:real|sim`, `order:<visit>`, `memo:<visit>`, `return:<memo>`). The same body after a timeout or reload therefore reuses the same key, and a changed body gets a new key. Keys are cleared on success or on a 4xx rejection. Proofs carry their own key inside the persisted `PendingProof`.

2.6 Tenant scoping and RBAC

- Every row carries `tenantId`, and every query filters by the JWT's `tid`. Foreign ids return `404`, never `403`, so existence never leaks (tests 6 and F1 build a second tenant through the API and probe every v1 and v2 read and write).
- Agents see only their own visits, orders, memos and proof files (`404` otherwise). Admins can read any record in the tenant, but mutating a visit (check-in, proof, order, memo) requires being the assigned agent, so an admin gets `403 NOT_YOUR_VISIT`.
- Permissions are declared per route. Adding a role (for example REGIONAL_MANAGER) is one entry in `ROLE_PERMISSIONS`.

2.7 Uploads: Vercel Blob or direct

`POST /v1/uploads/proof-token` checks that the visit is the caller's and is open, then builds a server-chosen object key `tenants/<slug>/visits/<visitId>/<ULID>.jpg`.

- **Blob mode** (when `BLOB_READ_WRITE_TOKEN` is set): returns a Vercel Blob client token scoped to exactly that pathname (10 min, ≤8 MB, image types only). The phone uploads with `put()` from `@vercel/blob/client`. On attach, the API requires the https URL to contain the object key. Bytes are not re-hashed server-side (documented decision).
- **Direct mode** (no token, as in local and self-hosted setups): the phone posts multipart to `/api/v1/uploads/direct` (≤4 MB). The API validates the type, key shape, tenant and open visit, recomputes SHA-256 and upserts `StoredFile` (bytea), so a retry is a no-op. It returns `url=/api/v1/files/:id`. `GET /v1/files/:id` streams the file: authenticated, tenant-scoped, agents limited to their own visits, `Cache-Control: private, immutable`, and the SHA-256 as ETag.

2.8 Outbound webhooks

Events are rows in `OutboxEvent` written in the same transaction as the change. The payload is exactly the public body `{id:"evt_<ULID>", type, tenantId:<slug>, createdAt, data}`. The dispatcher (`webhooks/dispatcher.ts`) has three triggers: the post-commit kick, an opportunistic `maybeKick` on `GET /admin/home`, `/admin/dashboard` and `/admin/webhooks/deliveries` (at most once per 10 s per instance), and the cron route. Each request is a POST with headers `X-JewelRoute-Event`, `-Delivery`, `-Timestamp` and `-Signature: sha256=HMAC_SHA256(secret, "<timestamp>.<raw body>")`, a 10 s timeout and `redirect:"manual"`. Endpoint secrets (`whsec_…`) are stored encrypted with AES-256-GCM under `APP_ENCRYPTION_KEY`. There are 17 event types (`packages/shared WEBHOOK_EVENT_TYPES`); v3 added `order.confirmed`, `order.pending_approval`, `order.settled`, `order.cancelled` and `memo.acknowledged`. The same outbox rows are the activity feed (§2.11). Pipeline and retry ladder: Figure 9.

2.9 Public acknowledgement page

Creating a memo returns `ack.url = APP_PUBLIC_URL/ack/<token>`. The token is 24 random bytes stored only as SHA-256 and valid for 72 h. The agent shows it as a QR code. `/ack/[token]` renders `AckView`, which calls the public endpoints with plain `fetch` (no cookies, no refresh, no redirect). The retailer sees the exact pieces and can tap *Confirm receipt*, or read a 6-digit code to the agent. The code is `HMAC(APP_SECRET, memoId:token) mod 10⁶`, shown only while pending and stored only as a hash. Five wrong codes lock code entry with `423 ACK_LOCKED`; the QR path still works. Either confirmation path now emits `memo.acknowledged` (v3), so the receipt shows up in the activity feed and

on the sample's custody row (`CustodyEventRow.ack`). While receipt is pending, the retailer page re-reads every 5 s, so a code typed on the agent's phone flips it to "confirmed".

2.10 Credit-aware orders and the global lock order (v3)

`OrdersService.createInTx()` is shared by `POST /orders` and `POST /memos/:id/convert`. Inside the transaction it prices every line at the current rate, then asks `CreditService.position(tid, shopId, {amountPaise: total, lock: true})`, which takes the shop row `FOR UPDATE` and sums: opening outstanding (`Shop.openingOutstandingPaise`) + unsettled CONFIRMED/APPROVED orders = **outstanding**; OUT memo items at the shop = **memo exposure**; their sum = **exposure**. The arithmetic is the pure `creditPosition()` in `packages/shared` (6 known-answer tests). The order is written `CONFIRMED` or `PENDING_APPROVAL` with `approvalRequired`, `approvalReasons` and the frozen `creditSnapshot`, and emits `order.created` plus `order.confirmed` or `order.pending_approval`. Because two orders at the same shop serialise on the shop lock, they cannot both slip under the limit. Settle and cancel take the same shop lock, so receivables stay consistent.

Write path	Locks, in this order (Memo → Visit → SamplePiece by id → Shop → Order)
Memo convert (sell from memo)	Memo → Visit (if acting on one) → Pieces (custody move) → Shop (credit rule) → new Order
Order create	Visit → Shop
Approve / reject a memo conversion	`lockConversion`: Memo → its sold Pieces, then Shop → Order (guarded update), then un-sell on reject
Approve / reject / settle / cancel (plain order)	Shop → Order
Memo create / return	Memo or Visit → Pieces

Taking Shop before Memo let reject ‖ convert on the same memo deadlock (11 of 12 runs in the re-review probe, surfacing as `409 CONCURRENT_UPDATE`); `test/v3.test.ts` V13 (6 rounds) is green with the order above. Source: `docs/DECISIONS.md` › Lock order, `docs/brain/clusters/orders-pricing.md`.

2.11 One activity model (v3)

`OutboxService.emit()` derives refs from each payload with the pure `eventRefs()` (`core/event-refs.ts`; the SQL back-fill in migration `…0300` and the batched `db:backfill-activity-refs` script use the same rules) and stores them as indexed columns: `shopId`, `agentId`, `visitId`, `orderId`, `memoId` (also `sourceMemoId`) and the GIN-indexed `pieceIds[]`. `ActivityService` (`core/activity.ts`) is the only code that turns rows into `ActivityItem {id, type, kind, title, amountPaise, actor, refs, detail, createdAt}`, resolving ref labels in one grouped query per record type. `kind` is one of `visit · proof · order · memo · custody · ack · rate · webhook · system`; `detail` carries the evidence (photo url, from → to labels, approval reasons, verification, distance, ack method). Home, Dashboard and every record's `activity` array are the same projection with a filter; the web renders them with one `ActivityFeed` component. The event's `evt_…` id is the webhook payload id, the ActivityItem id and `CustodyEventRow.eventId`.

3 API design

3.1 Principles

Principle	How it shows up in the code
Contract-first	`docs/CONTRACT.md` is the single document both apps build against (shapes, conventions, changelog). `CLAUDE.md` and `AGENTS.md` point agents at it, and the v2 section is marked "implemented; additive".
One envelope	`{ok,data}` or `{ok:false,error{code,message,details},requestId}` on every route, including errors raised by Nest itself (§2.4).
Integers for money and weight	Money is integer paise (₹1 = 100 paise; `BigInt` in Postgres, number in JSON). Weight is integer milligrams. Making charge is either paise per gram (`PER_GRAM`) or basis points of gold value (`PERCENT`, 1200 = 12%). Rounding is half-up to whole paise. GST (3%) is computed once on the order subtotal, not per line.
Server-authoritative pricing	`price()` (`core/rates.ts`) uses the shared `priceDesign`. Orders read rates once per transaction (one `DISTINCT ON` query) and freeze them in `OrderItem` rows and `Order.goldRateSnapshot`. A purity with no rate gives `409 GOLD_RATE_MISSING`, never a ₹0 price.
IST business dates	The business day is `istDate()` (Asia/Kolkata), sent as `YYYY-MM-DD` and stored as a Postgres `date`. Timestamps are ISO-8601 UTC. `plannedTime` is an IST wall-clock `HH:MM` (CHECK in SQL). "Today", "overdue", check-in day and MISSED are all IST.
Idempotency	Required on money and custody creates, optional on visit transitions; mechanics in §2.5.
Append-only ledgers	`GoldRate`, `CustodyEvent` and `OrderItem` have `BEFORE UPDATE OR DELETE` triggers that raise (the demo seed purges via a transaction-local flag). The current rate is the latest row. Custody is the ordered event chain with a gap-free `seq` per piece.
Transactional outbox	An event exists if and only if its state change committed. It doubles as the one activity feed: indexed refs per event, one projection (`ActivityService`, §2.11), cursor paging.
Approval is a rule, not a default (v3)	Orders are `CONFIRMED` unless `CREDIT_LIMIT` or `ORDER_THRESHOLD` trips inside the create transaction; the reasons and the `CreditPosition` are frozen on the order (`approval`). Clients may preview but never decide.
One lock order (v3)	Memo → Visit → SamplePiece (id order) → Shop → Order on every write path (§2.10).
No existence leaks	Cross-tenant or not-yours returns `404` (§2.6).
Resumable flows	`GET /visits/:id` returns a server-derived `resume` block (`nextStep`, `canCheckIn`, `canSell`, `canComplete`, `blockedBy`, `pendingAcks`), so a reload lands on the right step.
Gap-free document numbers	`nextDocNumber()` upserts `DocSequence` under a row lock, giving `ORD-2026-00042` and `MEMO-2026-00017` (per tenant per year), piece tags `AJ-S-000123` (kind `TAG`, year 0) and shop codes `SH-0001` (kind `TAG`, year 1).

3.2 Versioning: v1 and additive v2

Every route is under `/v1`. "v2" is not a URL prefix. It is the Attio-style IA added on 21 Oct 2026 (CONTRACT.md § "v2"): new record endpoints (`/admin/home`, `/admin/search`, `/admin/*/:id` records, `/field/visits`, `/field/catalog`, `/field/profile`) and enriched row shapes (`AdminVisitRow`, `ShopRow`, `PieceRow`, `CustodyEventRow`, `AttentionItem`, `AuditRow`). They are strict supersets: v1 fields are kept (for example `ShopRow.piecesHere` and `AuditRow.actorName`), so v1 clients keep working. Until today, `admin-v2/*/data.ts` fell back to composing v1 routes when a v2 route was missing. Those fallbacks were removed in commit `4d26473`, so the admin web now calls the v2 routes directly.

v3 (same day, CONTRACT.md § v3 and the "api, v3" changelog entries) is also additive at the URL level, with one deliberate behaviour change: `Order.status` no longer has `SUBMITTED`. Legacy rows were re-evaluated by the credit rule in migration `…0300` (→ `PENDING_APPROVAL` or `CONFIRMED`, snapshot marked `legacy:true`); the enum value was re-added in `…0400` only for rollback and is forbidden by CHECK `Order_no_submitted`. `GET /admin/orders?status=SUBMITTED` is still accepted as an alias of `PENDING_APPROVAL`. New: seven routes (credit ×2, convert-preview, settle, cancel, settings

GET/PATCH), `Order.approval / settledAt / cancelledAt / cancellationReason`, `Shop.openingOutstandingPaise`, `ShopRow.credit / upcomingVisit / lastOrder`, `ActivityItem.kind / refs / detail`, `CustodyEventRow.ack / order`, and `GET /admin/activity` filters with `limit`/`before` paging (without either, still the bare v1 array of 100).

3.3 Conventions at a glance

Purity	`K24 K22 K18 K14` (24/22/18/14 karat)
Visit status	`PLANNED CHECKED_IN ACTIVE COMPLETED MISSED CANCELLED`
Verification	`VERIFIED LOW_ACCURACY SIMULATED`
Custodian	`VAULT AGENT SHOP SOLD LOST`
Order status	`PENDING_APPROVAL CONFIRMED APPROVED REJECTED CANCELLED` (+`DRAFT` unused; `SUBMITTED` forbidden)
Approval reasons	`CREDIT_LIMIT ORDER_THRESHOLD`
Custody reason	`INTAKE ISSUE_TO_AGENT MEMO_HANDOVER MEMO_RETURN RETURN_TO_VAULT CONVERTED_TO_SALE SALE_REJECTED` (+`ADJUSTMENT` unused)

Memo status	`ACTIVE PARTIALLY_RETURNED RETURNED CONVERTED_TO_ORDER`; display adds `DUE_SOON OVERDUE`
Memo item	`OUT RETURNED CONVERTED`
Delivery	`PENDING SUCCEEDED FAILED DEAD`
Activity kind	`visit proof order memo custody ack rate webhook system`
Geofence	inside ⇔ distance ≤ radius + min(accuracy, 50 m); >75 m accuracy or unknown accuracy → `LOW_ACCURACY`
Status codes	creates 201, actions 200 (`@HttpCode(200)`)

3.4 Error catalogue

Code	HTTP	When (source)
`VALIDATION_FAILED`	400	zod parse failure on body or query; invalid enum filters on list endpoints; bad delivery status filter; bad activity or delivery cursor; empty or unknown settings keys.
`INVALID_JSON` · `BAD_REQUEST`	400	Malformed JSON body · other Nest 400s.
`IDEMPOTENCY_KEY_REQUIRED`	400	Missing key on `POST /orders`, `/memos`, `/memos/:id/return`, `/memos/:id/convert`.
`INVALID_DATE`	400	Assignment date before today (IST).
`INVALID_DUE_DATE`	400	Memo due date outside today…+90 days; an extension not after the current due date, in the past, or more than 90 days out.
`FILE_REQUIRED`	400	Direct upload without a `file` part.
`UNAUTHENTICATED`	401	No token on a protected route.
`TOKEN_EXPIRED` · `INVALID_TOKEN`	401	Access JWT expired (the web refreshes and retries) · signature or issuer invalid.
`INVALID_CREDENTIALS`	401	Wrong email or password (same message for an unknown email).
`NO_REFRESH_TOKEN` · `INVALID_REFRESH_TOKEN`	401	Refresh without a token · expired, revoked, reused or disabled account.
`INVALID_CRON_SECRET`	401	`/internal/*` without a valid `x-cron-secret` or Bearer `CRON_SECRET`.
`INVALID_SIGNATURE`	401	Demo receiver: signature does not verify or timestamp older than 5 min.
`FORBIDDEN`	403	The role lacks the route's permission (for example an agent calling `/admin/*`).
`NOT_YOUR_VISIT`	403	An admin tries to mutate an agent's visit (check-in, proof, order, memo).
`SIMULATION_NOT_ALLOWED`	403	`simulated:true` check-in while `Tenant.demoMode` is false.
`NOT_FOUND`	404	Unknown id, another tenant's id, or another agent's visit, order, memo or file (no existence leak); invalid or unknown ack token; Prisma P2025.
`ACK_EXPIRED`	410	Pending ack link older than 72 h.
`OUTSIDE_GEOFENCE`	409	Real check-in farther than radius + capped accuracy slack; `details {distanceM, radiusM}`.
`ALREADY_CHECKED_IN`	409	Check-in on a visit that is already open (also mapped from the partial unique index).

Code	HTTP	When (source)
ANOTHER_VISIT_OPEN	409	The agent has another CHECKED_IN or ACTIVE visit; details {visitId}.
VISIT_NOT_PLANNED · VISIT_NOT_TODAY	409	Check-in on a MISSED, CANCELLED or COMPLETED visit, or unassigning a non-planned visit · planned date is not today (IST).
VISIT_NOT_OPEN	409	Proof token, proof, order, memo or GET /visits/:id/credit on a visit that is not CHECKED_IN or ACTIVE.
VISIT_ALREADY_COMPLETED · PROOF_REQUIRED	409	Complete called twice · complete without at least one proof.
VISIT_SHOP_MISMATCH	409	Return or convert with an explicit visitId at a different shop than the memo.
ALREADY_ASSIGNED · AGENT_INACTIVE	409	Same agent, shop and date already planned (unique index) · assigning to a deactivated agent.
DESIGN_INACTIVE · GOLD_RATE_MISSING	409	An order line references a deactivated design · no current rate for a purity (orders, memo valuation, conversion).
ORDER_ALREADY_DECIDED	409	Approve or reject on an order already APPROVED or REJECTED (guarded update); details {status, decidedBy, decidedAt} name the winner of a lost race.
ORDER_NOT_PENDING	409	Approve or reject on a CONFIRMED, CANCELLED or DRAFT order; details {status}.
ORDER_ALREADY_SETTLED · ORDER_NOT_SETTLEABLE	409	Settle twice · settle an order that is not CONFIRMED or APPROVED.
ORDER_NOT_CANCELLABLE	409	Cancel of a settled, decided or memo-converted order; details {status, settled, sourceMemoId?}.
PROOF_REQUIRED_BEFORE_SALE	409	Order, memo, return, convert or convert-preview on a visit with no arrival photo; nothing written, key rolled back.
PIECE_NOT_SOLD	409	Un-selling a rejected conversion finds a piece no longer SOLD (custody expected-holder check).
PIECE_NOT_IN_YOUR_CUSTODY	409	Memo create with a piece the agent does not hold; details {pieceIds, tagCodes}.
PIECE_ALREADY_ON_MEMO	409	Partial unique index MemoItem_one_out_per_piece hit by a concurrent memo.
PIECE_NOT_ON_MEMO · PIECE_NOT_AT_SHOP	409	Return or convert of a piece that is not OUT on this memo · not physically at the memo's shop.
PIECE_NOT_IN_VAULT · PIECE_NOT_WITH_AGENT	409	Issue of a non-vault piece · return-to-vault of pieces not all with the same agent.
NO_PIECES	409	Custody move with an empty piece list.
MEMO_CLOSED · ALREADY_ACKNOWLEDGED	409	Extend on a RETURNED or CONVERTED memo · reissuing an ack link after acknowledgement.
SHOP_CODE_TAKEN · SKU_TAKEN · EMAIL_TAKEN	409	Unique per-tenant code, SKU or email.
IDEMPOTENCY_IN_PROGRESS · IDEMPOTENCY_RETRY	409	Duplicate key while the original has no stored response yet · race where the key row vanished.
CONCURRENT_UPDATE	409	Prisma P2034, serialization failure or deadlock; safe to retry.
PAYLOAD_TOO_LARGE	413	JSON body over 1 MB (multipart limits are 4 MB direct, 8 MB token).
IDEMPOTENCY_KEY_REUSED	422	Same key, different body or different user.
INVALID_OBJECT_KEY · OBJECT_NOT_FOUND · INVALID_URL · CHECKSUM_MISMATCH	422	Proof key outside tenants/<slug>/visits/<id>/ · no stored file in direct mode · blob URL does not contain the key · direct-mode SHA-256 differs from the claimed checksum.
UNSUPPORTED_MEDIA_TYPE	422	Direct upload that is not jpeg, png, webp or heic.
ACK_CODE_INVALID	422	Wrong 6-digit retailer code (attempt counted).
ACK_LOCKED	423	Five or more wrong codes; the retailer must confirm on their phone or the agent reissues the link.
RATE_LIMITED	429	Throttler limits (§2.3).
INTERNAL · SIMULATED_FAILURE	500	Unhandled error (logged) · demo receiver token starting with fail-.
UNHEALTHY	503	GET /v1/health cannot reach the database.

4 Complete endpoint reference

89 routes generated from the controller decorators in `apps/api/src/**`. Paths are shown without the browser prefix `/api`. **Role**: `field` = FIELD_AGENT or ADMIN; `admin` = ADMIN only; `auth` = any signed-in user; `public` = no auth. **Tx** = runs in one Prisma transaction (`maxWait` 10 s, `timeout` 20 s). **Key** = Idempotency-Key (req = required, opt = optional). Every successful write except auth, internal dispatch and the demo receiver also triggers a post-commit dispatcher run. Rows marked **v3** are new or changed since the previous edition.

4.1 Auth

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
POST	`/v1/auth/login`	public	Email and password (bcrypt, dummy-hash on unknown user; optional `tenantSlug`). Issues a 15 min access JWT and a 30 day refresh token, sets `jr_at`/`jr_rt`, returns `{user, accessToken, refreshToken}`. 10/min per IP+email.	—	—	RefreshToken
POST	`/v1/auth/refresh`	public	Rotates the refresh token (cookie or `body.refreshToken`). Reuse of a revoked token revokes the user's whole family. Sets new cookies. 30/min.	conditional claim (`updateMany`)	—	RefreshToken
POST	`/v1/auth/logout`	public	Revokes the presented refresh token and clears both cookies.	—	—	RefreshToken
GET	`/v1/auth/me`	auth	Current user plus tenant `{id, slug, name, demoMode}`; drives role routing and the demo-mode "Simulate arrival" control.	read	—	—

4.2 Field: today, visits, check-in, proofs, notes, catalog

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/field/today?date=`	field	The caller's route for an IST date (default today): non-cancelled visits by sequence, progress counts, `nextVisitId`, `openVisit` (any date, so a visit left open yesterday is surfaced), bag count and value at current rates, and open memos (count, outstanding value, overdue).	read	—	—
GET	`/v1/field/visits?scope=recent|upcoming`	field	Own visits: `recent` = the last 14 days including all of today (non-cancelled); `upcoming` = the next 7 days, as `AdminVisitRow` with outcome counts.	read	—	—
GET	`/v1/field/catalog?q&category&collection&purity&minWeightMg&maxWeightMg`	field	Active designs priced at current rates, facets (categories, collections, purities), current rates, and `availablePieces` from the caller's bag.	read	—	—
GET	`/v1/field/profile`	field	Own user, bag (`PieceRow[]`), open memos by due date, and 7-day stats (visits completed, order value, memos open).	read	—	—

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/visits/:id`	field (own; admin any)	Visit workspace: `resume` block (now with `returnedCount`), visit summary, shop, proofs, notes (with authors), orders and memos created in the visit, `returns` (memo returns made during this visit, grouped per action), shop context (5 previous orders, active memos at the shop, last completed visit) and the agent's bag.	read	—	—
POST	`/v1/visits/:id/geofence-check`	field	`{lat,lng,accuracyM?}` → server distance (haversine), radius, accuracy-aware verdict and verification label, plus a human message. No state change. 200.	none	—	—
POST	`/v1/visits/:id/check-in`	field (assigned)	PLANNED → CHECKED_IN. Under a row lock it checks: not already open, planned for today (IST), inside the fence (or `simulated:true` on a demo tenant, which records SIMULATED), and no other open visit. Stores position, accuracy, distance and verification. 200.	key opt (web sends); tx + `FOR UPDATE`; partial unique index backstop	`visit.checked_in`	Visit, OutboxEvent, IdempotencyKey
POST	`/v1/visits/:id/proofs`	field (assigned)	Attaches an uploaded photo. The key must be under `tenants/<slug>/visits/<id>/`. Direct mode checks the stored SHA-256; blob mode requires an https URL containing the key. Dedupes on `(visitId, checksum)`. The first proof moves CHECKED_IN → ACTIVE. Records capture position and distance from check-in. 201.	key opt (web sends one per photo); tx + lock	`visit.proof_uploaded` (not on dedupe)	VisitProof, Visit, OutboxEvent, IdempotencyKey
POST	`/v1/visits/:id/notes`	field (readable)	Appends a free-text note (≤4000 chars) visible to the manager. Any visit status. An admin may also add notes. 201.	none; single insert	—	VisitNote
POST	`/v1/visits/:id/complete`	field (assigned)	Open → COMPLETED with optional summary and position. Requires at least one proof. The event carries proof, order and memo counts and values. 200.	key opt (web sends none); tx + lock	`visit.completed`	Visit, OutboxEvent
GET	`/v1/visits/:id/catalog?…`	field	Same as the field catalog, but `availablePieces` is the visit agent's bag (so an admin reading a visit sees the agent's bag).	read	—	—
GET	`/v1/visits/:id/credit?amountPaise=`	field (own open visit)	**v3.** Bare `CreditPosition` of the visit's shop for a prospective order total: limit, opening + unsettled = outstanding, memo exposure, exposure, available (`null` = no limit), projected exposure, `requiresApproval`, `reasons`, `asOf`. Not open → `409 VISIT_NOT_OPEN`; another agent's visit → 404. Called by the field order review on every total change.	read	—	—

4.3 Field: uploads and files

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
POST	`/v1/uploads/proof-token`	field	`{visitId, contentType, sizeBytes≤8 MB}` for the caller's open visit. Returns `{mode:"blob", clientToken, pathname}` (Vercel Blob token scoped to the pathname, 10 min) or `{mode:"direct", pathname, uploadUrl:"/api/v1/uploads/direct", maxBytes:4 MB}`. 201.	none	—	—
POST	`/v1/uploads/direct`	field	Multipart `file` + `pathname`. Validates type, key shape and tenant, and that the visit is the caller's and open. Recomputes SHA-256 and stores the bytes. Returns `{url:/api/v1/files/:id, objectKey, checksumSha256, sizeBytes, contentType}`.	upsert on `(tenantId, objectKey)`, so a retry is a no-op	—	StoredFile
GET	`/v1/files/:id`	auth	Streams a direct-mode file. Tenant-scoped; agents may open only files under their own visits' keys. `Cache-Control: private, max-age=31536000, immutable`, ETag = SHA-256.	read	—	—

4.4 Field: orders

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
POST	`/v1/orders`	field (assigned, visit open)	`{visitId, items[{designId, quantity 1..500}] (≤100), notes?}`. Locks the visit and rejects inactive designs. Requires an arrival photo on the visit (`409 PROOF_REQUIRED_BEFORE_SALE`). Prices every line at the current gold rate inside the transaction, freezes `OrderItem` rows and `goldRateSnapshot`, numbers `ORD-YYYY-NNNNN`. **v3:** runs the credit rule with the shop locked and writes `CONFIRMED` or `PENDING_APPROVAL` with `approval {required, reasons, position}`. Returns `{order}`. 201.	**key req**; tx; locks Visit → Shop; `clientOpId` unique backstop returns the existing order	`order.created` + `order.confirmed` or `order.pending_approval`	Order, OrderItem, DocSequence, AuditLog, OutboxEvent ×2, IdempotencyKey
GET	`/v1/orders/:id`	field (own; admin any)	One order view (lines with frozen prices, totals, snapshot, approval, decision, settlement). No web caller (the field reads orders inside `GET /visits/:id`).	read	—	—

4.5 Field: memos and retailer acknowledgement

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
POST	`/v1/memos`	field (assigned, visit open)	`{visitId, pieceIds[1..100], dueDate (today…+90), notes?}`. Needs the arrival photo. Numbers `MEMO-YYYY-NNNNN` and moves each piece AGENT → SHOP (`MEMO_HANDOVER`) through the custody ledger. Values pieces at the current rate (`MemoItem` OUT, memo total), then issues a 72 h ack token and code. Returns `{memo, ack:{url, expiresAt}}`. 201.	**key req**; tx; pieces locked `FOR UPDATE ORDER BY id`; partial unique index → `PIECE_ALREADY_ON_MEMO`	`sample.custody_changed`, `memo.created`	Memo, MemoItem, SamplePiece, CustodyEvent, DocSequence, AuditLog ×2, OutboxEvent ×2, IdempotencyKey

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/memos/:id`	field (own; admin any)	Memo view: stored and display status, items with pieces, extensions, ack state. No web caller since the admin fallback was removed.	read	—	—
POST	`/v1/memos/:id/return`	field (own memo; admin any)	`{pieceIds, visitId?}`. Moves OUT pieces SHOP → the memo's agent (`MEMO_RETURN`), marks items RETURNED and recomputes the memo status. The move is attributed to the caller's open visit at that shop (or an explicit open visit at the same shop). 200.	**key req**; tx; memo row `FOR UPDATE`	`sample.custody_changed`, `memo.returned`	MemoItem, Memo, SamplePiece, CustodyEvent, AuditLog ×2, OutboxEvent ×2, IdempotencyKey
POST	`/v1/memos/:id/extend`	field (own)	`{dueDate, reason?}`. The new date must be after the current one, ≥ today and ≤ +90 days. Writes `MemoExtension` and clears `overdueNotifiedAt` (re-arms `memo.overdue`). 200. Called by the field **Extend** drawer.	tx + lock; no key	`memo.extended`	MemoExtension, Memo, AuditLog, OutboxEvent
GET	`/v1/memos/:id/convert-preview?pieceIds=a,b&visitId=`	field (own memo)	**v3 (P7).** Read-only preview of the convert below for the same pieces: one line per piece priced at today's rate (`price()`, GST once on the subtotal) with its memo value at handover, totals, rate snapshot, and `credit` from the shared rule with those pieces already out of memo exposure. Writes nothing. `409 PIECE_NOT_ON_MEMO`; with `visitId` also `VISIT_NOT_OPEN` · `VISIT_SHOP_MISMATCH` · `PROOF_REQUIRED_BEFORE_SALE`.	read	—	—
POST	`/v1/memos/:id/convert`	field (own)	`{pieceIds, visitId?}`: sells OUT pieces. Locks Memo → Visit → Pieces, moves SHOP → SOLD (`CONVERTED_TO_SALE`), marks items CONVERTED *before* the credit rule (no double count), then `createInTx` (Shop lock, one line per piece at its own net weight, today's rate, `sourceMemoId`) → **CONFIRMED or PENDING_APPROVAL**; recomputes the memo status. Returns `{memo, order}`. 200. Called by the field **Sell from memo** drawer.	**key req**; tx; order `clientOpId = convert:<key>`	`sample.custody_changed`, `order.created`, `order.confirmed|pending_approval`, `memo.converted`	SamplePiece, CustodyEvent, Order, OrderItem, DocSequence, MemoItem, Memo, AuditLog, OutboxEvent ×4
POST	`/v1/memos/:id/ack/verify`	field (own)	`{code: 6 digits}`. Compares against the HMAC hash in constant time. A match sets ACKNOWLEDGED (method OTP); a miss increments attempts, and the 5th miss locks. 200, `422 ACK_CODE_INVALID` or `423 ACK_LOCKED`. 20/min.	tx + memo lock	`memo.acknowledged` (v3)	Memo, AuditLog, OutboxEvent
POST	`/v1/memos/:id/ack/link`	field (own)	Reissues the ack link: new token, code hash and 72 h expiry, attempts reset. The old link stops working. 409 if already acknowledged. 200.	tx + lock	—	Memo

Where the ack events go: since v3 both confirmation paths (agent code, retailer page) emit `memo.acknowledged` `{memoId, number, method, ackAt, by:'retailer', shop, agent, visitId?, pieces[]}`, so the receipt appears in the activity feed (kind `ack`) and on the handover row of each piece's custody ledger.

4.6 Admin: home, dashboard, activity, search, settings

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/admin/home?date=`	admin	v2 Home: metrics (orders today, visits, samples at retailers, overdue memos, pending approvals), map (shops plus check-ins), server-built `attention[]` (memo overdue or due today, missed visit at its planned slot, missing or simulated proof, order pending approval with its reasons, failing webhook), 40 activity items, agents, rates. `pendingApprovals` counts PENDING_APPROVAL orders; order value metrics exclude REJECTED and CANCELLED.	read; `maybeKick`	—	—
GET	`/v1/admin/dashboard?date=`	admin	v1 dashboard: the same metrics, map, activity and agents plus `exceptions` (pending orders, overdue memos, missed visits) and the day's visits. No web caller now (still listed on `/developers`).	read; `maybeKick`	—	—
GET	`/v1/admin/activity?shopId&agentId&visitId&orderId&memoId&pieceId&limit&before`	admin	**v3.** The one activity stream: outbox events (no `ping`) filtered by indexed refs, newest first, projected to `ActivityItem {kind, title, amountPaise, actor, refs{shop, agent, visit, order, memo, pieces[]}, detail}`. Without `limit`/`before`: bare array of 100 (v1 shape). With either: `{items, nextCursor?}`, keyset on (createdAt, id), limit 1–200 (default 50); a bad cursor gives 400. Called by every `ActivityFeed` (Home and six record types).	read	—	—
GET	`/v1/admin/search?q=`	admin	⌘K search: up to 5 each of shops, pieces (tag, design), orders, memos, agents and designs, case-insensitive.	read	—	—
GET	`/v1/admin/settings`	admin	**v3.** `{approvalThresholdPaise: number|null, demoMode}`. Seed threshold ₹10,00,000.	read	—	—
PATCH	`/v1/admin/settings`	admin	**v3.** Same keys; threshold a positive integer or `null` (off); empty or unknown keys → 400. Affects every later credit check, not existing orders. Called by Settings › Approvals.	tx	— (audit `tenant.settings`)	Tenant, AuditLog

4.7 Admin: shops

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/admin/shops?q&agentId&status`	admin	`ShopRow[]`: assigned agent, last and next visit, open orders (PENDING_APPROVAL + CONFIRMED + APPROVED, 30 days), memo exposure, samples here, status ACTIVE / MEMO_OVERDUE / DORMANT / NEW, and v3 `credit: CreditPosition`, `upcomingVisit`, `lastOrder` (grouped queries, ≤1000).	read	—	—

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/admin/shops/:id`	admin	Shop record: row, visits in the last 30 days and upcoming, orders, memos, pieces at the shop now, activity mentioning the shop.	read	—	—
POST	`/v1/admin/shops`	admin	Creates a shop (geofence 10–2000 m, default 100; credit limit and v3 `openingOutstandingPaise`). The code is auto-generated as `SH-NNNN` if omitted; a duplicate gives 409 `SHOP_CODE_TAKEN`. 201.	tx	—	Shop, AuditLog (DocSequence if auto code)
PATCH	`/v1/admin/shops/:id`	admin	Partial update (the code is immutable), including `assignedAgentId` or `null`; the agent is validated. Omitted fields are no longer reset to create defaults (v3 fix). Before and after values are audited.	tx	—	Shop, AuditLog
GET	`/v1/admin/shops/:id/credit?amountPaise=`	admin	**v3.** Bare `CreditPosition` for any shop, optionally for a prospective amount. No web caller (the shop record uses `ShopRow.credit`).	read	—	—

4.8 Admin: visits and assignments

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/admin/visits?date|from&to&agentId&shopId&status&proof`	admin	`AdminVisitRow[]` (default: today), with proof state, duration and outcome (orders, memos, returns), ≤1000. CSV filters are validated.	read	—	—
GET	`/v1/admin/visits/:id`	admin	Field visit detail plus `row`, `audit` (visit, its orders and memos), `activity`, and `custody` (every move made during the visit, with location and proof).	read	—	—
POST	`/v1/admin/assignments`	admin	`{agentId, shopId, date, sequence?, plannedTime?}` creates a PLANNED visit (sequence = max + 1). The date must be ≥ today and the agent active. A duplicate agent, shop and date gives 409 `ALREADY_ASSIGNED`. 201.	tx; unique index	—	Visit, AuditLog
DELETE	`/v1/admin/assignments/:visitId`	admin	Removes a stop from the route: PLANNED → CANCELLED (guarded update); any other status gives 409 `VISIT_NOT_PLANNED`.	tx; guarded update	—	Visit, AuditLog

4.9 Admin: agents

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/admin/agents`	admin	AgentRow per field agent: today planned, completed and in progress; current and next visit; bag count and value; open memos (count, value, overdue); orders today; last check-in (grouped queries).	read	—	—
GET	`/v1/admin/agents/:id`	admin	Agent record: profile, today's visit rows, bag (`PieceRow[]`), open memos, last 30 orders, activity.	read	—	—
POST	`/v1/admin/agents`	admin	Creates a FIELD_AGENT (bcrypt password ≥8 chars); a duplicate email gives 409 `EMAIL_TAKEN`. 201. Called by **Add agent** on the Agents list.	tx	—	User, AuditLog
PATCH	`/v1/admin/agents/:id`	admin	Updates name, phone, home area, password or active. Deactivation revokes all refresh tokens. Called by **Deactivate/Reactivate agent** on the agent record (body `{active}`).	tx	—	User, RefreshToken, AuditLog

4.10 Admin: orders

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/admin/orders?status&from&to&agentId&shopId`	admin	Order views, newest first, ≤500. `status` is a CSV of the v3 values (`SUBMITTED` accepted as an alias of PENDING_APPROVAL). Date filters are IST days on `createdAt`.	read	—	—
GET	`/v1/admin/orders/:id`	admin	Order record: order, source visit row (verification, distance), audit history, and `activity` for the order (outbox events by aggregate or `data.orderId`).	read	—	—
POST	`/v1/admin/orders/approve`	admin	`{orderIds ≤200}` batch approve. Duplicate ids are collapsed; each order runs through the same `decide()` in its own transaction, so one failure (`NOT_FOUND`, `ORDER_ALREADY_DECIDED`, `ORDER_NOT_PENDING`) never rolls back the others. Returns per-order `results[{id, ok, number?, status?, error?}]`, `approved` and `failed`. 200. Called by the Orders list bulk approve.	one tx per order; guarded update	`order.approved` ×N	Order, AuditLog, OutboxEvent
POST	`/v1/admin/orders/:id/approve`	admin	**v3:** PENDING_APPROVAL → APPROVED via `updateMany … WHERE status='PENDING_APPROVAL'`, recording the decider and time; the order now counts as outstanding. Lost race → `409 ORDER_ALREADY_DECIDED` naming who decided; any other status → `409 ORDER_NOT_PENDING`. 200.	tx; Memo → Pieces (if converted) → Shop lock; guarded update	`order.approved`	Order, AuditLog, OutboxEvent

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
POST	`/v1/admin/orders/:id/reject`	admin	`{reason}` PENDING_APPROVAL → REJECTED with `rejectionReason`; the agent sees the reason. **v3:** for a memo conversion (`sourceMemoId`) it **un-sells**: pieces SOLD → the memo's shop (`SALE_REJECTED`), memo items CONVERTED → OUT, memo status recomputed (e.g. CONVERTED_TO_ORDER → ACTIVE), memo exposure restored. Response unchanged `{order}`; the event adds `sourceMemoId` and `unsold {memoId, memoNumber, memoStatus, pieceIds}`. 200.	tx; lock order as approve; guarded update; deferred trigger `Order_dead_unsells_pieces`	`order.rejected` (+ `sample.custody_changed` when un-selling)	Order, AuditLog, OutboxEvent; when un-selling also SamplePiece, CustodyEvent, MemoItem, Memo
POST	`/v1/admin/orders/:id/settle`	admin	**v3.** No body. Marks a CONFIRMED or APPROVED order paid (`settledAt`); it stops counting towards the shop's outstanding. Returns `{order, credit}` (position after). Again → `409 ORDER_ALREADY_SETTLED`; other statuses → `409 ORDER_NOT_SETTLEABLE`. 200.	tx; Shop lock; guarded update	`order.settled`	Order, AuditLog, OutboxEvent
POST	`/v1/admin/orders/:id/cancel`	admin	**v3.** `{reason?}`. CONFIRMED or PENDING_APPROVAL, not settled and not converted from a memo → CANCELLED (`cancelledAt`, `cancellationReason`); else `409 ORDER_NOT_CANCELLABLE`. 200.	tx; Shop lock; guarded update	`order.cancelled`	Order, AuditLog, OutboxEvent

4.11 Admin: memos

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/admin/memos?filter=active|due_today|overdue|returned|all&shopId&agentId`	admin	Memo views plus `pieceCount` and `issuedAt`, ≤300 (open filters by due date; returned and all by newest).	read	—	—
GET	`/v1/admin/memos/:id`	admin	Memo record: memo, chronological `custody` rows for its pieces from the vault issue that preceded the handover onward, source visit row, audit history, and `activity` for the memo.	read	—	—
POST	`/v1/admin/memos/:id/extend`	admin	Same service as the field extend (an admin may extend any memo in the tenant). 200.	tx + lock	`memo.extended`	MemoExtension, Memo, AuditLog, OutboxEvent

4.12 Admin: designs, pieces and custody

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/admin/designs`	admin	All designs (including inactive) priced at current rates, each with all its pieces.	read	—	—
POST	`/v1/admin/designs`	admin	Creates a design (SKU, category, collection, purity, gross and net mg, making charge, stones, other components, image URLs). Duplicate SKU gives 409 `SKU_TAKEN`. 201.	tx	—	JewelryDesign, AuditLog

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
PATCH	`/v1/admin/designs/:id`	admin	Partial update, including `active` (an inactive design cannot be ordered).	tx	—	JewelryDesign, AuditLog
POST	`/v1/admin/designs/:id/pieces`	admin	Intake: creates 1–50 physical pieces in the VAULT with tags `AJ-S-NNNNNN`, `custodySeq` 1 and an `INTAKE` custody event each. 201.	tx; **not idempotent**	— (no outbox event)	SamplePiece, CustodyEvent, DocSequence, AuditLog
GET	`/v1/admin/pieces?q&status&custodianType&agentId&shopId&designId`	admin	`PieceRow[]` with derived status IN_VAULT / WITH_AGENT / ON_MEMO / OVERDUE / SOLD / LOST, open memo, last move, value, and for agent-held pieces a `location`: the agent's open visit, else their last check-in, else "In field bag". ≤1000.	read	—	—
GET	`/v1/admin/pieces/:id`	admin	Piece record: row (with location), design, full immutable ledger as `CustodyEventRow[]` (from/to labels, actor, visit location, proof, memo, outbox event id), and the piece's `activity`.	read	—	—
GET	`/v1/admin/pieces/:id/history`	admin	v1 ledger (seq, from/to labels, reason, actor, memo number). No web caller now.	read	—	—
POST	`/v1/admin/pieces/issue`	admin	`{pieceIds ≤200, agentId}` VAULT → AGENT (`ISSUE_TO_AGENT`); the agent must be active. A piece not in the vault gives 409 `PIECE_NOT_IN_VAULT`. 200.	tx; pieces `FOR UPDATE`	`sample.custody_changed`	SamplePiece, CustodyEvent, AuditLog, OutboxEvent
POST	`/v1/admin/pieces/return-to-vault`	admin	`{pieceIds}` AGENT → VAULT (`RETURN_TO_VAULT`); all pieces must be with the same agent. 200.	tx; locks	`sample.custody_changed`	SamplePiece, CustodyEvent, AuditLog, OutboxEvent

4.13 Admin: gold rates and audit

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/admin/gold-rates`	admin	The current rate per purity (with who set it) and the last 60 history rows.	read	—	—
POST	`/v1/admin/gold-rates`	admin	`{purity, ratePaisePerGram}` appends a rate, effective now. Existing orders are untouched (frozen snapshot); new prices, bags and memo valuations use it immediately. 201.	tx; append-only table	— (no webhook)	GoldRate, AuditLog
GET	`/v1/admin/audit?entityType&entityId`	admin	≤200 audit rows, newest first, with actor reference, human `summary` and resolved references (`core/audit-summary.ts`). No web caller (records embed their audit rows).	read	—	—

4.14 Webhooks and integrations

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/admin/webhooks/event-types`	admin	The 17 event types with descriptions and example envelopes.	read	—	—
GET	`/v1/admin/webhooks`	admin	Endpoints with `secretLast4` and 24 h stats (succeeded; failed + dead).	read	—	—
POST	`/v1/admin/webhooks`	admin	`{url (https in prod), description?, events[] or ["*"]}`. Generates a `whsec_` secret, stores it encrypted, and returns it once. 201.	tx	—	WebhookEndpoint, AuditLog
GET	`/v1/admin/webhooks/deliveries?endpointId&status&eventType&cursor&limit`	admin	Without `cursor`/`limit`: the latest 100 as a bare array (v1 shape). With either: keyset pages `{items, nextCursor}` on (createdAt, id) desc, limit ≤200 (default 50); the cursor is opaque.	read; `maybeKick`	—	—
GET	`/v1/admin/webhooks/deliveries/:id`	admin	Delivery with the signed payload and per-attempt log (headers actually sent, status, latency, error, response body ≤2000 chars).	read	—	—
POST	`/v1/admin/webhooks/deliveries/:id/retry`	admin	Manual retry: resets to PENDING and claims in one `UPDATE … RETURNING`, then attempts immediately. If another worker holds the lock, it waits up to about 10 s for that attempt. 200.	atomic reset+claim	— (re-sends existing)	WebhookDelivery, WebhookAttempt
GET	`/v1/admin/webhooks/received`	admin	Last 50 requests received by the built-in demo receiver (headers, body, `signatureOk`).	read	—	—
PATCH	`/v1/admin/webhooks/:id`	admin	Updates URL, description, events or `active`.	tx	—	WebhookEndpoint, AuditLog
DELETE	`/v1/admin/webhooks/:id`	admin	Soft delete: deactivates the endpoint and moves PENDING and FAILED deliveries to DEAD ("Endpoint deleted"). History is kept.	tx	—	WebhookEndpoint, WebhookDelivery, AuditLog
POST	`/v1/admin/webhooks/:id/rotate-secret`	admin	New secret, effective immediately, returned once. 200.	tx	—	WebhookEndpoint, AuditLog
GET	`/v1/admin/webhooks/:id/secret`	admin	Decrypts and returns the secret. Audited as `webhook.reveal_secret` (a GET that writes an audit row).	tx (audit)	—	AuditLog
POST	`/v1/admin/webhooks/:id/test`	admin	Emits a `ping` targeted at this endpoint only, creates its delivery, and delivers synchronously. Returns the delivery detail. 200.	tx, then sync attempt	`ping`	OutboxEvent, WebhookDelivery, WebhookAttempt

4.15 Public

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/public/memo-ack/:token`	public (token)	Retailer view: tenant, shop, agent, memo number, due date, pieces (tag, name, purity, weights, value), total, ack state and the 6-digit code while pending. Expired and pending gives 410 `ACK_EXPIRED`. 30/min.	read	—	—
POST	`/v1/public/memo-ack/:token`	public (token)	`{confirm:true}` marks ACKNOWLEDGED (method QR_CONFIRM) via a guarded update. Repeats return the same result. Audit row with actor `null` and the retailer's IP. 10/min.	tx; guarded update (idempotent)	`memo.acknowledged` (v3)	Memo, AuditLog, OutboxEvent

4.16 Ops

Method	Path	Role	What it does	Idempotency / tx	Events	Tables written
GET	`/v1/health`	public	`SELECT 1`. Returns `{status, db, time, version}` or 503 `UNHEALTHY`. Not throttled, not logged.	read	—	—
POST	`/v1/internal/dispatch`	cron secret	One dispatcher run: emit `memo.overdue` (once per memo), mark past PLANNED visits MISSED, fan out the outbox, deliver up to 50 due deliveries. Returns run stats.	each step in its own tx	`memo.overdue`	Memo, Visit, OutboxEvent, WebhookDelivery, WebhookAttempt
GET	`/v1/internal/dispatch`	cron secret	Same as POST (Vercel cron issues GET; scheduled daily 03:00 UTC = 08:30 IST).	as above	`memo.overdue`	as above
POST	`/v1/internal/seed`	cron secret	Wipes and recreates the demo tenant `aurum` (Aurum Jewels), including the demo webhook endpoint.	purge flag for append-only tables	—	all tenant tables
POST	`/v1/dev/webhook-receiver/:token`	public	Built-in receiver. Verifies the signature exactly as a customer would (timestamp within 5 min) and stores the hit. Tokens starting with `fail-` always answer 500 (to demo retries). 120/min.	—	—	ReceivedWebhook

5 Data model

25 Prisma models in `apps/api/prisma/schema.prisma`, nine migrations (`…0000_init` to `…0800`; v3 is `…0300_v3_credit_activity`, `…0400_v3_expand_enums`, `…0500_v3_sale_guards`, `…0600_v3_validate_no_submitted`, plus two concurrent OrderItem indexes). v3 added columns, not tables; `CreditPosition` is computed, never stored except as the frozen `Order.creditSnapshot`. Every business row carries `tenantId`. Money columns are `BigInt` paise, weights are `Int` milligrams, business dates are `@db.Date`.

Figure 2 (relationship diagram) is on the landscape page at the end of this section.

5.1 Tables by area

Table	Key fields	Uniques and indexes · notes
Tenancy and identity		
Tenant	`slug`, `name`, `demoMode` (allows SIMULATED check-ins), v3 `approvalThresholdPaise?`	`slug` unique; threshold null or > 0
User	`email`, `name`, `role` FIELD_AGENT|ADMIN, `passwordHash` (bcrypt), `active`, `phone`, `homeArea`	(tenantId, email) unique; (tenantId, role)
RefreshToken	`tokenHash` (SHA-256), `expiresAt`, `revokedAt`, `userAgent`	`tokenHash` unique; rotation by revocation
Field operations		
Shop	`code`, `name`, owner, phone, address, area, city, `lat/lng`, `geofenceRadiusM` (default 100), `creditLimitPaise` (0 = no limit), v3 `openingOutstandingPaise`, `assignedAgentId?`	(tenantId, code) unique; (tenantId, city); (tenantId, assignedAgentId); both credit columns ≥ 0; row locked by every exposure-changing write
Visit	`shopId`, `agentId`, `plannedDate` (date), `plannedTime?` HH:MM, `sequence`, `status`, check-in lat/lng/accuracy/distance, `verification`, `startedAt`, `completedAt`, completion position, `summary`, `assignedById`	(tenant, agent, shop, plannedDate) unique; partial unique "one open per agent"; indexes by date, agent+date, shop+date
VisitProof	`objectKey`, `url`, content type, size, `checksumSha256`, `capturedAt`, capture and check-in positions, `distanceM`	(visitId, checksumSha256) unique, so the same photo is attached once
VisitNote	`visitId`, `authorId`, `body`	—
Catalog and pricing		
GoldRate	`purity`, `ratePaisePerGram`, `effectiveFrom`, `source` MANUAL, `createdById`	append-only; (tenant, purity, effectiveFrom); current = latest ≤ now
JewelryDesign	`sku`, name, category, collection, `purity`, gross and net mg, `makingChargeType/Value`, `stoneValuePaise`, `otherComponents` JSON, `imageUrls[]`, `active`	(tenantId, sku) unique; (tenant, category)
Custody		
SamplePiece	`designId`, `tagCode`, gross and net mg (per piece), `custodianType`, `custodianAgentId?`, `custodianShopId?`, `custodySeq`	(tenantId, tagCode) unique; indexes by custodian type, agent and shop
CustodyEvent	`pieceId`, `seq`, from and to (type, agent, shop), `reason` (v3 adds `SALE_REJECTED`: SOLD → shop), `memoId?`, `visitId?`, `actorId`, `outboxEventId?`, `occurredAt`	append-only; (pieceId, seq) unique, gap-free; indexes by time, visit, memo
Commerce		
Order	`number`, shop, agent, `visitId?`, `status` (v3: PENDING_APPROVAL · CONFIRMED · APPROVED · REJECTED · CANCELLED; DRAFT unused), `clientOpId`, subtotal, GST, total (paise), `goldRateSnapshot` JSON, notes, `sourceMemoId?`, decided by and at, `rejectionReason`; v3 `approvalRequired`, `approvalReasons[]`, `creditSnapshot` JSON, `settledAt?`, `cancelledAt?`, `cancellationReason?`	(tenant, clientOpId) and (tenant, number) unique; (tenant, status, createdAt); (tenant, visitId); (tenant, shop, createdAt); v3 (tenant, shop, status)

Table	Key fields	Uniques and indexes · notes
OrderItem	design, `pieceId?`, sku, name, purity, qty, net mg, `goldRateId`, rate, gold, making, stone, other, unit and line totals	append-only frozen price snapshot; indexes on `pieceId` and `orderId` (for the sale guards and un-sell)
Memo	`number`, shop, agent, `visitId?`, `status`, `clientOpId`, `dueDate`, `totalValuePaise`, `ackStatus`, `ackTokenHash`, `ackCodeHash`, `ackAttempts`, `ackAt`, `ackMethod`, `ackTokenExpiresAt`, `overdueNotifiedAt`, `closedAt`	(tenant, clientOpId), (tenant, number), `ackTokenHash` unique; (tenant, status, dueDate)
MemoItem	`memoId`, `pieceId`, `valuePaise` (exposure at handover), `status` OUT|RETURNED|CONVERTED, `returnedAt`	partial unique: one OUT line per piece
MemoExtension	`fromDueDate`, `toDueDate`, `reason`, `actorId`	history of extensions; the view exposes `extended: true`
Integration		
WebhookEndpoint	`url`, description, `secretEncrypted` (AES-256-GCM), `secretLast4`, `events[]`, `active`	soft-deleted via `active=false`
OutboxEvent	`id` evt_ULID, `type`, aggregate type and id, `actorId`, `payload` (the exact public body), `fannedOutAt`, `targetEndpointId?` (ping); v3 ref columns `shopId?`, `agentId?`, `visitId?`, `orderId?`, `memoId?`, `pieceIds[]`	(tenant, createdAt); (fannedOutAt); (tenant, aggregate); v3 (tenant, <ref>, createdAt) for each ref; GIN on `pieceIds`
WebhookDelivery	`endpointId`, `eventId`, `status`, `attempts`, `nextAttemptAt`, `lockedUntil`, last status code, latency, error, `deliveredAt`	(endpointId, eventId) unique; (status, nextAttemptAt)
WebhookAttempt	`attemptNo`, status code, latency, error, response body, `requestHeaders` JSON	one row per HTTP attempt
ReceivedWebhook	endpoint token, delivery and event ids, headers, body, `signatureOk`	demo receiver log
CreditPosition (v3, computed)	`shopId`, `creditLimitPaise`, `openingOutstandingPaise`, `unsettledOrdersPaise`, `outstandingPaise`, `memoExposurePaise`, `exposurePaise`, `availableCreditPaise|null`, `overLimit`, `approvalThresholdPaise|null`, `orderAmountPaise`, `projectedExposurePaise`, `requiresApproval`, `reasons`, `asOf`	Not a table. `CreditService.position()` → shared `creditPosition()`; frozen into `Order.creditSnapshot`; returned as `Order.approval.position`, `ShopRow.credit` and by both credit routes
Platform		
StoredFile	`objectKey`, content type, size, `sha256`, `bytes` (bytea), `uploadedById`	(tenantId, objectKey) unique; direct-mode proofs
AuditLog	`actorId?` (null = retailer via public link), `action`, entity type and id, `before`/`after` JSON, `requestId`, `ip`	(tenant, entityType, entityId); (tenant, createdAt)
IdempotencyKey	`key`, `userId`, `fingerprint`, `responseStatus`, `responseBody`	PK (tenantId, key)
DocSequence	`kind` ORD|MEMO|TAG, `year`, `next`	PK (tenantId, kind, year); row-locked upsert

5.2 Database-enforced invariants

Hand-written in `migrations/20261021000000_init/migration.sql` (after the generated DDL), `…0200_v2_attio_ia` and the v3 migrations `…0300`/`…0500`/`…0600`. These hold even if application code has a bug.

Constraint	Kind	Rule
`Visit_one_open_per_agent`	partial unique	`(tenantId, agentId) WHERE status IN ('CHECKED_IN','ACTIVE')`: at most one open visit per agent. Backs the `ANOTHER_VISIT_OPEN` check against races (test 1b).
`MemoItem_one_out_per_piece`	partial unique	`(pieceId) WHERE status = 'OUT'`: a piece can be out on at most one memo (test 4, F9).
`SamplePiece_custodian_consistent`	CHECK	AGENT ⇔ `custodianAgentId` set; SHOP ⇔ `custodianShopId` set. Every piece has exactly one custodian.
`SamplePiece_weights` · `JewelryDesign_weights`	CHECK	net > 0, gross ≥ net, `custodySeq` ≥ 1 · making charge and stone value ≥ 0.

Constraint	Kind	Rule
`Order_totals` · `OrderItem_values`	CHECK	`total = subtotal + gst`, all ≥ 0 · `quantity > 0`, `lineTotal = unitPrice × quantity`.
`GoldRate_positive` · `MemoItem_value`	CHECK	rate > 0 · item value ≥ 0.
`Shop_geo`	CHECK	radius > 0, lat within ±90, lng within ±180.
`CustodyEvent_seq` · `Memo_ack_attempts`	CHECK	`seq ≥ 1` · `ackAttempts ≥ 0`.
`Visit_plannedTime_format`	CHECK	`plannedTime` null or `^([01][0-9]|2[0-3]):[0-5][0-9]$`.
`GoldRate_append_only` · `CustodyEvent_append_only` · `OrderItem_append_only`	trigger	`BEFORE UPDATE OR DELETE … jr_forbid_mutation()` raises `check_violation`. The only escape is DELETE with the transaction-local `jewelroute.allow_purge = on` used by the demo seed (test 8).
`Order_pending_has_reason` · `Order_approval_reasons_known`	CHECK (v3)	PENDING_APPROVAL ⇒ `approvalRequired` and ≥1 reason · reasons ⊆ {CREDIT_LIMIT, ORDER_THRESHOLD}.
`Order_settled_only_when_owed` · `Order_cancelled_at`	CHECK (v3)	`settledAt` only on CONFIRMED/APPROVED · CANCELLED ⇔ `cancelledAt` set.
`Order_no_submitted`	CHECK (v3)	No row may be SUBMITTED (enum value kept only for rollback); added NOT VALID, validated in `…0600`.
`Shop_credit_nonneg` · `Tenant_approval_threshold_positive` · `OutboxEvent_pieceIds_not_null`	CHECK (v3)	limit and opening ≥ 0 · threshold null or > 0 · refs array never null.
`SamplePiece_sold_has_live_order` · `Order_dead_unsells_pieces`	deferred constraint trigger (v3)	At COMMIT: a SOLD piece must appear on an OrderItem of an order that is not REJECTED or CANCELLED. Fires when a piece becomes SOLD and when a selling order is rejected or cancelled, so a rejection that does not un-sell (or a cancel of a conversion) cannot commit. Deferred because convert moves the piece before inserting the order.
Plain uniques	unique	Per-tenant email, shop code, SKU, tag code, order and memo number, `clientOpId`; (visit, checksum); (piece, seq); (endpoint, event); (agent, shop, date).

5.3 Relationship diagram

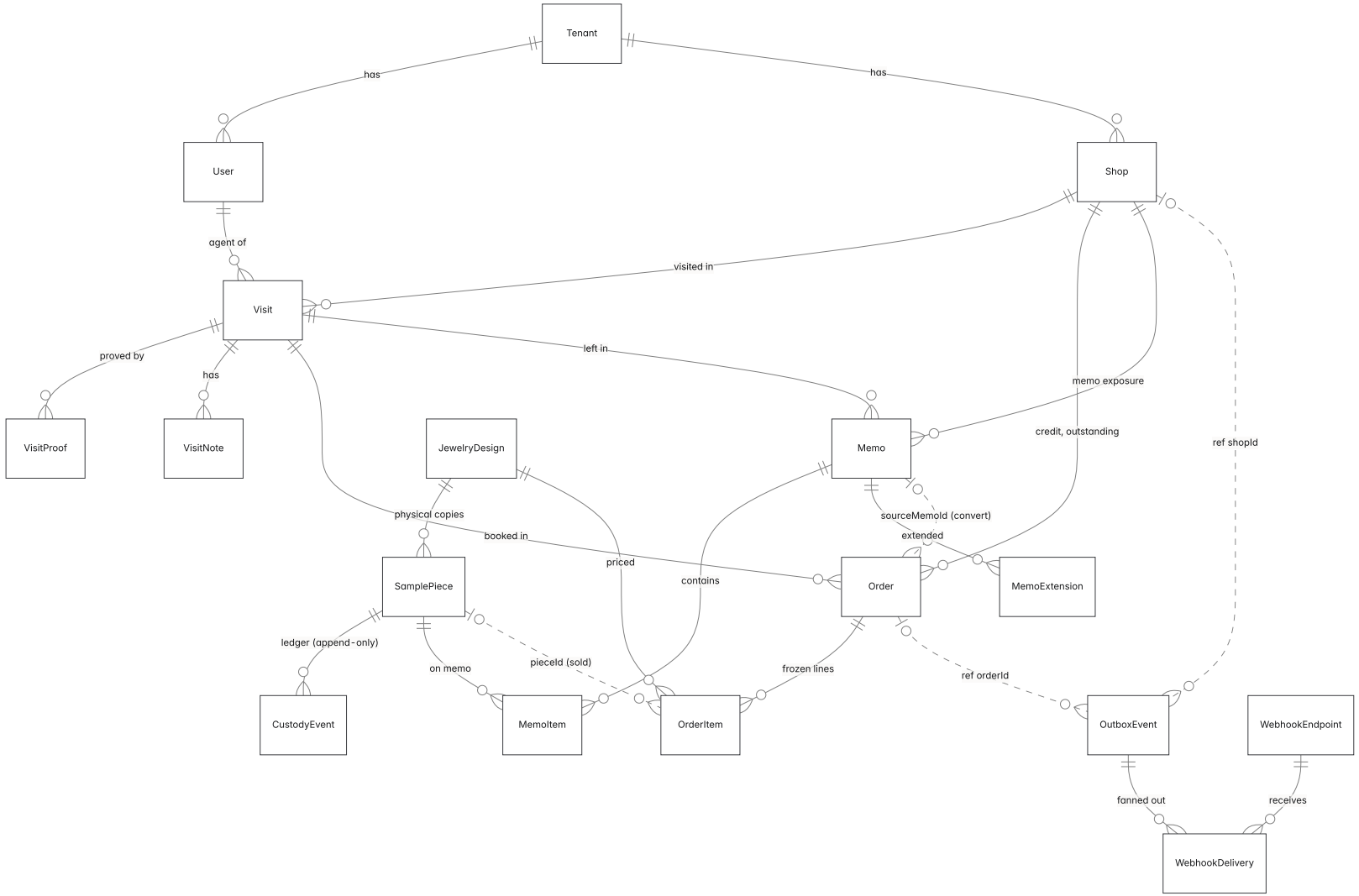


Figure 2 — Core relationships. Dashed lines are references without a foreign key: `Order.sourceMemoId`, `OrderItem.pieceId`, and two of the six v3 activity refs on `OutboxEvent`. Not shown: RefreshToken, GoldRate, WebhookAttempt, ReceivedWebhook, StoredFile, AuditLog, IdempotencyKey, DocSequence (see the table above).

6 Click-by-click data flow

Every user-facing action in the web app, traced from the component that handles it. Endpoints are shown as the browser calls them, after `/api`. "sync(x)" means `useSync()(x)` from `lib/sync.ts`, which invalidates every key in `AFFECTS[x]` (prefix match, so `["visit"]` also refreshes `["visit", id]`). The two most common lists: **V** (`everywhereVisit`) = today, visit, field-visits, home, dashboard, admin-visits, admin-visit, shop, shops, agent, agents, activity. **C** (`custody`) = credit, convert-preview, pieces, piece, profile, catalog, designs, memos, memo, shop, shops, agent, agents, today, visit, admin-visit, home, dashboard, activity. **D** (`order.decided`) = orders, order, home, dashboard, shop, shops, admin-visit, visit, activity, credit, convert-preview, agent, agents, today. §6.11–6.13 follow three v3 flows end to end across both apps.

6.1 Login

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
`/login`	**Sign in** (email + password form)	Mutation via `api()`; the button shows a spinner and the form is ignored while busy.	`POST /v1/auth/login`	bcrypt check; insert RefreshToken; `Set-Cookie: jr_at, jr_rt`	`qc.setQueryData(["me"], user)`; `router.replace` to `/admin` or `/field/today` (or `?next=` inside the role's area). 401 shows "That email and password don't match".
`/login`	**Sign in as Rahul (field agent)**	Same call with the demo credentials `rahul@aurum.test` / `Rahul@2026`; errors also toast.	`POST /v1/auth/login`	as above	→ `/field/today`
`/login`	**Sign in as Priya (manager)**	Same with `admin@aurum.test` / `Aurum@2026`.	`POST /v1/auth/login`	as above	→ `/admin`
Field Profile · admin user menu	**Sign out**	`logout()` in `lib/session.ts`, then `location.href="/login"`.	`POST /v1/auth/logout`	revoke refresh token; clear cookies	full page load (cache gone)

6.2 Field: Today and opening a visit

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
`/field/today`	(page load)	Query `["today"]`, polled every 60 s and on focus. Starts `watchPosition` (high accuracy, own 15 s timeout); distances to shops are computed locally for display only.	`GET /v1/field/today`	read	—
`/field/today`	**Open visit** / **Continue visit** (focus row, sticky bar), any route row, a map marker	Navigates to `/field/visits/:id`; query `["visit", id]` refetches on focus.	`GET /v1/visits/:id`	read; returns `resume.nextStep`, which picks the stage	—

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
`/field/today`	**Resume and complete** (banner when a visit from an earlier day is still open)	Navigates to that visit (from `openVisit`).	`GET /v1/visits/:id`	read	—
`/field/today`	Directions icon	Opens the maps directions URL in a new tab; no API.	—	—	—
`/field/today`	"In your bag / Out on memo" card	Navigates to `/field/profile`.	`GET /v1/field/profile`	read	—
`/field/today`	Location chip **Tap to retry**	Restarts geolocation; no API.	—	—	—
`/field/visits`	**Recent** / **Upcoming** tabs, a row	Query `["field-visits", scope]`; Recent is today's whole route plus the last 14 days, straight from the API. A row opens the visit.	`GET /v1/field/visits?scope=`	read	—

6.3 Field: geofence and check-in

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
Visit (PLANNED)	(automatic) geofence polling	Query `["geofence", id]` while a fix exists: every 5 s (not in background), plus 600 ms after the agent moves ≥15 m from the last checked position. The UI treats "inside" as true only if the server said so for a position within 25 m of the current fix.	`POST /v1/visits/:id/geofence-check`	none (pure computation)	local only; panel shows distance, "Verifying", "Location verified" or "Outside the zone"
Visit (PLANNED)	**Check in** (enabled only when the server says inside and `resume.canCheckIn ≠ false`)	Guarded single run. Body `{lat, lng, accuracyM}`; key `opKey.for("checkin:<id>:real","real")`, reused on network retry.	`POST /v1/visits/:id/check-in`	tx: lock visit, checks; Visit → CHECKED_IN (VERIFIED or LOW_ACCURACY); outbox `visit.checked_in`; idempotency row; post-commit dispatch	sync(`visit.checked_in`) = **V**. Errors: `OUTSIDE_GEOFENCE` toast + fence refetch; `ANOTHER_VISIT_OPEN` toast with "Open it"; `ALREADY_CHECKED_IN` → sync.
Visit (PLANNED, demo tenant)	**Simulate arrival (demo, recorded as simulated)** (also in the GPS-failure panel)	Same call with the shop's own lat/lng, `accuracyM:5`, `simulated:true`; key scope `checkin:<id>:sim`. Shown only when `me.tenant.demoMode`.	`POST /v1/visits/:id/check-in`	as above with verification SIMULATED (403 `SIMULATION_NOT_ALLOWED` if not a demo tenant)	sync(`visit.checked_in`) = **V**; Home attention shows "Simulated GPS check-in"
Visit (PLANNED)	**Retry location**, **Directions**	Restart `watchPosition` · external maps link.	—	—	—

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
Visit (blocked)	**Open that visit**	Shown when `resume.blockedBy = ANOTHER_VISIT_OPEN`; navigates to the open visit.	`GET /v1/visits/:id`	read	—

6.4 Field: arrival photo (proof)

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
Visit (UPLOAD_PROOF)	**Take arrival photo** (camera input) / **Upload** (file picker)	(1) Prepare: downscale to ≤1600 px, JPEG q 0.82 (strips EXIF), SHA-256, new idempotency key; saved to sessionStorage `jr:proof:<visitId>`. (2) Token. (3) Upload bytes with progress: blob `put()`, or XHR multipart. (4) Attach.	`POST /v1/uploads/proof-token` → blob store *or* `POST /v1/uploads/direct` → `POST /v1/visits/:id/proofs`	Direct: StoredFile upsert. Attach tx: VisitProof (dedupe by checksum), Visit CHECKED_IN → ACTIVE, outbox `visit.proof_uploaded`	sync(`visit.proof_uploaded`) = **V**; the stage advances to "Sell and complete"; toast "Arrival photo attached"
Visit (UPLOAD_PROOF)	**Retry upload**	Resumes the saved `PendingProof`: skips the upload if bytes are already stored and re-attaches with the **same key**. On `INVALID_OBJECT_KEY`, `OBJECT_NOT_FOUND`, `CHECKSUM_MISMATCH` or `INVALID_URL` it drops the stored-object marker and re-uploads the same bytes. A photo prepared before a reload comes back as "failed, retry".	same as above	replay returns the stored response; checksum dedupe prevents a second proof	as above; on `VISIT_NOT_OPEN` it clears the pending photo and syncs
Visit (UPLOAD_PROOF)	**Use a different photo**	New file → new prepare, new key.	as above	as above	as above
Visit workspace	Proof summary row	Opens the evidence drawer (photo, time, verification, distances, checksum). `<img>` loads via `/api/v1/files/:id` with cookies in direct mode.	`GET /v1/files/:id` (direct mode)	read	—

6.5 Field: visit workspace, notes, catalog, order

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
Visit workspace	**Add note** → **Save note**	Drawer; the draft survives reloads (`jr:note:<id>`). Guarded single run; no key.	`POST /v1/visits/:id/notes`	insert VisitNote	sync(`visit.note_added`) = visit, admin-visit, shop, activity

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
Visit workspace	**Browse catalog** · **Create order** · **Leave on memo**	Navigates to `/field/visits/:id/sale?mode=order&browse=1`, `?mode=order` or `?mode=memo`. The sale view loads the visit plus an unfiltered catalog (facets, rates).	`GET /v1/visits/:id`, `GET /v1/visits/:id/catalog`	read	—
Catalog tab `/field/catalog`	(page load); **Order for <shop>** when a visit is open	Browse-only catalog (`["catalog","browse",…]`); the button navigates to the open visit's sale page.	`GET /v1/field/catalog`, `GET /v1/field/today`	read	—
Sale: catalog	Search box, category chips (**All**, …), purity radio (**Any**/22K/…), **Clear filters**	Search is debounced 250 ms. Query key `["catalog", visitId, filters]` keeps the previous data while loading; stale after 30 s.	`GET /v1/visits/:id/catalog?q=&category=&purity=`	read	—
Sale: catalog	Design card (tap)	Opens the design drawer: photos, weights, full price breakdown from the list response, pieces in the bag. No extra call.	—	—	—
Sale: catalog	Card **+** quick add · drawer quantity −/+ · **Add to order** / **Update order** / **Remove from order**	Edits the local draft only (sessionStorage `jr:sale:<visitId>`, design id → qty). Disabled with a reason when the visit is not open or has no proof.	—	—	bottom bar shows a display-only preview (shared `orderTotals`)
Sale: catalog	**Review order**	Local step change to review.	—	—	—
Sale: review order	(automatic) **Credit check** card	Query `["credit","visit",id,total]` re-runs when the total changes; shows exposure against the limit and, if a rule would trip, why.	`GET /v1/visits/:id/credit?amountPaise=`	read (no lock)	—
Sale: review order	−/+ per line, notes, **Place order** or, when the check says approval is needed, **Send for approval** (later **Place order again** / **Retry**)	Guarded. Items sorted by design id; body `{visitId, items, notes}`; key `opKey.for("order:<visit>", JSON(body))`. `sendConfirmed()`: on a timeout or 5xx it re-sends with the same key ("Checking") before saying anything; if still unknown it keeps the draft and offers **Retry**. A 4xx clears the key.	`POST /v1/orders`	tx: idempotency row, lock Visit, proof check, price at current rates, lock Shop and run the credit rule → CONFIRMED or PENDING_APPROVAL, Order + OrderItems, DocSequence, AuditLog, outbox `order.created` + `order.confirmed|pending_approval`	sync(`order.created`) = credit, convert-preview, orders, order, visit, admin-visit, today, shop, shops, agent, agents, home, dashboard, activity, profile. Then "Order confirmed" or "Sent for approval".
Order booked	**Back to visit** · **Book another order**	Clears `orderDone` from the draft; navigates, or returns to browse.	—	—	—

6.6 Field: memo, retailer acknowledgement, return, complete

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
Sale	Mode tab **Order** / **Memo**	`router.replace(?mode=)`. Memo mode lists the agent's exact pieces from `visit.bag`. Draft pieces that left the bag are dropped automatically.	(cached `GET /v1/visits/:id`)	—	—
Sale: memo	Piece rows (checkbox), bag search, **Review memo**	Local draft (piece ids).	—	—	—
Review memo	Due back **3 days** / **5 days** / **7 days**, calendar (tomorrow...+90), remove ✕, notes	Local draft; default due date is today + 5.	—	—	—
Review memo	**Leave N pieces on memo**	Guarded. Body `{visitId, pieceIds (sorted), dueDate, notes}`; key `opKey.for("memo:<visit>", JSON(body))`.	`POST /v1/memos`	tx: Memo, custody AGENT → SHOP per piece (locks, CustodyEvent, seq + 1), MemoItems valued now, ack token and code, AuditLog, outbox `sample.custody_changed` + `memo.created`	sync(`memo.created`) = **C**; shows the ack screen. On `PIECE_NOT_IN_YOUR_CUSTODY` or `PIECE_ALREADY_ON_MEMO` it removes those pieces from the draft and syncs `piece.moved`.
Ack screen (agent)	QR code; **Copy link**; **Retailer page**; **Skip for now, back to visit**	Shows the QR of `ack.url`. Polls `["visit", id]` every 3 s until the memo shows ACKNOWLEDGED.	`GET /v1/visits/:id` (poll)	read	When ACKNOWLEDGED appears: sync(`memo.acknowledged`) = memos, memo, visit, admin-visit, shop, activity; "Custody confirmed"
Ack screen (agent)	6-digit code boxes (auto-submit on the 6th digit)	Guarded verify; clears the boxes on error.	`POST /v1/memos/:id/ack/verify`	tx: memo lock; match → ACKNOWLEDGED (OTP) + AuditLog; miss → attempts + 1, lock at 5	sync(`memo.acknowledged`); toasts for wrong or locked codes
Visit timeline	**Show QR again** (pending memo)	Reissues the link, then opens the ack drawer.	`POST /v1/memos/:id/ack/link`	new token, code hash, expiry; attempts reset	— (the drawer polls the visit)
`/ack/[token]` (retailer)	(page load)	Plain `fetch` (no cookies or refresh); renders pieces, total, code; handles invalid, expired (410) and error states.	`GET /v1/public/memo-ack/:token`	read	—
`/ack/[token]` (retailer)	**Confirm receipt**	POST `{confirm:true}`; shows the receipt.	`POST /v1/public/memo-ack/:token`	guarded update → ACKNOWLEDGED (QR_CONFIRM); AuditLog (actor null, IP)	The agent's 3 s poll and admin polls pick it up.

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
Visit workspace	**Return samples** (action list) or memo **Return** → tick pieces → **Return N pieces**	Guarded. One call per memo; key `opKey.for("return:<memo>", ids)`.	`POST /v1/memos/:id/return` (×memos)	tx: memo lock, custody SHOP → AGENT, MemoItem RETURNED, memo status, AuditLog, outbox ×2	sync(`memo.returned`) = **C**, also on failure ("some memos may have gone through"); the return then shows on the visit timeline from `GET /visits/:id returns`
Visit workspace, "Samples at this shop"	Memo **Sell** → tick pieces (preview appears)	Drawer `SellFromMemoDrawer`. Each selection change queries `["convert-preview", memo, ids, visit]`; only a preview for exactly this selection enables the button ("Checking price and credit" meanwhile).	`GET /v1/memos/:id/convert-preview?pieceIds=&visitId=`	read: lines at today's rate, totals, credit net of the chosen pieces	—
Sell drawer	**Book order for N pieces** or **Send N pieces for approval** → **Done**	Guarded; body `{pieceIds (sorted), visitId}`; key `opKey.for("convert:<memo>", ids)`. Shows the server-created order; title "Order confirmed" or "Sent to your manager for approval".	`POST /v1/memos/:id/convert`	tx: Memo → Visit → Pieces locks, custody SHOP → SOLD, MemoItem CONVERTED, Shop lock + credit rule, Order + OrderItems (`sourceMemoId`), memo status, AuditLog, outbox ×4	sync(`memo.converted`) = **C** + orders, order
Visit workspace, "Samples at this shop"	Memo **Extend** → date (suggests +7 days, ≤ today+90) and reason → **Extend to <date>**	Drawer `ExtendMemoDrawer`; guarded; no key.	`POST /v1/memos/:id/extend`	MemoExtension, due date, re-arm overdue, AuditLog, outbox `memo.extended`	sync(`memo.extended`) = memos, memo, home, dashboard, shop, visit, admin-visit, activity
Visit workspace	**Complete visit** → drawer (summary) → **Complete visit**	Disabled without a proof. Attaches the current fix if any; no key; `VISIT_ALREADY_COMPLETED` is treated as success.	`POST /v1/visits/:id/complete`	tx: lock, proof ≥ 1, Visit → COMPLETED, outbox `visit.completed`	sync(`visit.completed`) = **V** + profile; `router.push("/field/today")`

6.7 Admin: Home, search, visits and routes

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
`/admin` Home	(page load)	Query `["home", date]` polled every 10 s. The Activity column is `ActivityFeed` (infinite query, 20 per page, polled 10 s). The sidebar rate strip reads `["gold-rates"]`.	`GET /v1/admin/home`, `GET /v1/admin/activity?limit=20`, `GET /v1/admin/gold-rates`	read; opportunistic dispatcher kick (≤1 per 10 s)	—
Home · every record's Activity tab	Activity **Agent** filter · **Load more** · a row or a linked ref	Filter changes the key (`?agentId=`); Load more fetches `before=nextCursor`; refs link to the shop, agent, visit, order, memo or piece record.	`GET /v1/admin/activity?…&before=`	read	—

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
Home	Date **‹** / **›** / **Back to today**	Changes the `date` in the key.	`GET /v1/admin/home?date=`	read	—
Home	Attention item, activity row, check-in row, agent row, map marker, **All visits** / **View all**	Navigates to the record (`/admin/visits/:id`, `/memos/:id`, `/orders/:id`, `/shops/:id`, `/agents/:id`, `/integrations`).	record GET (polled 15–20 s)	read	—
Any admin page	⌘K / Search box; result; "Go to" items	Debounced 150 ms, ≥2 chars, key `["search", q]`, stale 15 s; selecting navigates.	`GET /v1/admin/search?q=`	read	—
Home · Visits list · Agents list · Agent record (**Assign visit**, **Add stop**) · Shop record	**Assign visit** → dialog (shop, agent, date) → **Assign visit**	Loads shop options and agents, plus a route preview for the chosen agent and date; blocks a duplicate locally.	`GET /v1/admin/shops`, `GET /v1/admin/agents`, `GET /v1/admin/visits?date&agentId` → `POST /v1/admin/assignments`	tx: Visit PLANNED (sequence max + 1), AuditLog; no webhook	sync(`visit.assigned`) = **V**; the agent's Today shows it on the next poll or focus
`/admin/visits`	Filters, row, bulk **Copy row**	Query `["admin-visits", range]` polled every 15 s; rows open the record; copy is clipboard only.	`GET /v1/admin/visits?…`, `GET /v1/admin/agents`	read	—
Visit record · Agent record route	**Remove from route** (trash, with `confirm()`)	Only for PLANNED visits.	`DELETE /v1/admin/assignments/:visitId`	Visit → CANCELLED, AuditLog	sync(`visit.assigned`) = **V**
`/admin/agents`	**Add agent** → name, work email, phone, home area, temporary password (≥8) → **Add agent**	`useMutation`; navigates to the new agent's record.	`POST /v1/admin/agents`	User FIELD_AGENT (bcrypt), AuditLog	sync(`agent.saved`) = agents, agent, home, dashboard
Agent record	**Deactivate agent** / **Reactivate agent** (icon) → confirm	Mutation with `{active}`. A deactivated agent can no longer sign in or be assigned (Assign is disabled).	`PATCH /v1/admin/agents/:id`	User.active; on deactivate, all refresh tokens revoked; AuditLog	sync(`agent.saved`)
Visit record	**Copy link**, tabs	Clipboard; record query polled every 15 s.	`GET /v1/admin/visits/:id`	read	—

6.8 Admin: orders, memos, samples, catalog, gold rate

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
Order record · order preview (list)	**Approve order** / **Approve** (only PENDING_APPROVAL)	`useWrite` mutation; the button shows "Approving…". The Overview shows "Credit check at booking" from `order.approval`.	`POST /v1/admin/orders/:id/approve`	locks, guarded update PENDING_APPROVAL → APPROVED, AuditLog, outbox `order.approved`	sync(`order.decided`) = **D**; a lost race toasts "Already approved by …"
Order record · preview **Reject…** (`?reject=1`)	**Reject** → reason (≥3 chars) → **Reject order**	Mutation with `{reason}`. For a memo conversion the dialog loads the memo and warns "Rejecting returns N pieces to MEMO-… at <shop> — they go back on memo, not to the vault."	`POST /v1/admin/orders/:id/reject`	→ REJECTED + reason, AuditLog, outbox `order.rejected`; conversion: un-sell (custody SALE_REJECTED, items OUT, memo status)	sync(`order.decided`), plus `order.unsold` = **C** + orders, order for a conversion
Order record (CONFIRMED / APPROVED, unpaid)	**Mark as settled** → confirm	Mutation, no body.	`POST /v1/admin/orders/:id/settle`	Shop lock, `settledAt`, AuditLog, outbox `order.settled`	sync(`order.settled`) = orders, order, shop, shops, credit, convert-preview, home, dashboard, activity, agent
Order record (not settled, not from a memo)	**Cancel** → optional reason → **Cancel order**	Mutation with `{reason?}`.	`POST /v1/admin/orders/:id/cancel`	Shop lock, → CANCELLED, AuditLog, outbox `order.cancelled`	sync(`order.cancelled`)
`/admin/orders`	Status tabs (Pending approval, Confirmed, Approved, Unsettled, Rejected, Cancelled); select rows → bulk approve → **Approve**	One batch call with the selected ids; if any fail, the dialog stays open with per-order results.	`POST /v1/admin/orders/approve`	each order through `decide()` in its own tx: APPROVED, AuditLog, outbox `order.approved`	sync(`order.decided`)
Memo record	**Extend due date** → calendar, reason → **Extend to <date>**	Mutation.	`POST /v1/admin/memos/:id/extend`	MemoExtension, due date, re-arm overdue, AuditLog, outbox `memo.extended`	sync(`memo.extended`) = memos, memo, home, dashboard, shop, visit, admin-visit, activity
`/admin/samples` (bulk) · sample record · design sheet	**Issue to agent** → pick agent → **Issue to <agent>**	Only vault pieces can be selected for issue.	`POST /v1/admin/pieces/issue`	tx: custody VAULT → AGENT, CustodyEvent, AuditLog, outbox `sample.custody_changed`	sync(`piece.moved`) = **C**
Agent record	**Issue samples** → tick vault pieces → **Issue samples**	Loads the vault list for the dialog.	`GET /v1/admin/pieces?custodianType=VAULT` → `POST /v1/admin/pieces/issue`	as above	sync(`piece.moved`) on settle
Samples · sample record · design sheet · agent bag	**Return to vault** / **Return N to vault**	Only pieces in agents' bags can be selected.	`POST /v1/admin/pieces/return-to-vault`	custody AGENT → VAULT, outbox `sample.custody_changed`	sync(`piece.moved`) = **C**

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
`/admin/catalog`	**New design** / design tile → sheet → **Add design** / **Save design**	Create or patch.	`POST /v1/admin/designs` · `PATCH /v1/admin/designs/:id`	JewelryDesign, AuditLog	sync(`design.saved`) = designs, catalog, pieces
Design sheet	Count → **Add to vault**	Intake of N pieces.	`POST /v1/admin/designs/:id/pieces`	N SamplePieces in VAULT + INTAKE events, DocSequence, AuditLog (no webhook)	sync(`design.saved`, `piece.moved`)
`/admin/settings` Gold rate	**Set rate** → modal (preview of affected designs and pieces) → **Set 22K to ₹X/g**	`GoldRateSettings` (`admin-v2/sales/gold-rate.tsx`), `useWrite`.	`POST /v1/admin/gold-rates`	append GoldRate, AuditLog (no webhook)	sync(`gold_rate.set`) = gold-rates, convert-preview, catalog, designs, pieces, piece, home, dashboard, profile, shops, shop, agents, agent, activity
`/admin/settings?tab=approvals`	Approval threshold (₹, empty = Off) → **Save**	`useWrite`; rupees parsed to paise without floats; the credit-limit rule is shown read-only.	`GET` / `PATCH /v1/admin/settings`	Tenant.approvalThresholdPaise, AuditLog `tenant.settings`	sync(`settings.saved`) = settings, credit, convert-preview, shops, shop, me

6.9 Admin: shops and assigned agents

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
`/admin/shops`	**New shop** → sheet (geofence map preview) → **Create shop**	Client validation (code required, radius 20–1000 m, ₹ to paise without floats).	`POST /v1/admin/shops`	Shop, AuditLog	sync(`shop.saved`) = shops, shop, home, dashboard, today; navigates to the new shop
Shop record	Pencil **Edit shop** → **Save shop**	Patch with the same form.	`PATCH /v1/admin/shops/:id`	Shop, AuditLog (before and after)	sync(`shop.saved`)
Shop record	"Assigned agent" select	Immediate mutation on change (empty value → `null`).	`PATCH /v1/admin/shops/:id` `{assignedAgentId}`	Shop, AuditLog	sync(`shop.saved`, `agent.saved`)
`/admin/shops`	Select rows → **Assign agent** → agent / **Remove assignment**	N PATCH calls via `allSettled`.	`PATCH /v1/admin/shops/:id` ×N	as above	sync(`shop.saved`, `agent.saved`) on settle

6.10 Admin: webhooks and integrations

Screen	Control	What the browser does	Endpoint(s)	Server-side effect	Refreshes afterwards
`/admin/integrations`	(page load)	Queries: webhooks (15 s), event types (once), deliveries as an infinite query of 50-row cursor pages (loaded pages refetched every 5 s), the selected delivery (5 s), received (5 s).	`GET /v1/admin/webhooks`, `…/event-types`, `…/deliveries?limit=50&cursor=`, `…/received`	read; deliveries GET kicks the dispatcher	—
Integrations	**Add endpoint** → URL, description, events → **Add endpoint**	Mutation; then a one-time secret modal (**I've saved it**).	`POST /v1/admin/webhooks`	WebhookEndpoint (secret encrypted), AuditLog	sync(`webhook.changed`) = webhooks, deliveries, delivery, received, home
Endpoint panel	**Reveal** / **Hide**	Fetches the secret and hides it again after 30 s; no cache.	`GET /v1/admin/webhooks/:id/secret`	AuditLog `webhook.reveal_secret`	—
Endpoint panel	**Rotate** → **Rotate secret**	Confirm modal; shows the new secret once.	`POST /v1/admin/webhooks/:id/rotate-secret`	new encrypted secret, AuditLog	sync(`webhook.changed`)
Endpoint panel	**Send test event** (needs Enabled)	Mutation; opens the returned delivery's detail (polled 5 s).	`POST /v1/admin/webhooks/:id/test`	outbox `ping` targeted at this endpoint, delivery, synchronous signed attempt, WebhookAttempt	sync(`webhook.changed`)
Endpoint panel	Enabled switch · **Edit URL** → **Save** · events → **Save events**	PATCH with the changed field.	`PATCH /v1/admin/webhooks/:id`	WebhookEndpoint, AuditLog	sync(`webhook.changed`)
Endpoint panel	Trash → **Delete endpoint**	Confirm modal; deselects the endpoint.	`DELETE /v1/admin/webhooks/:id`	deactivate; PENDING/FAILED → DEAD; AuditLog	sync(`webhook.changed`)
Deliveries table · delivery detail	**Retry** (FAILED or DEAD rows)	Mutation; toast "Retry sent".	`POST /v1/admin/webhooks/deliveries/:id/retry`	atomic reset + claim, immediate signed attempt, WebhookAttempt	sync(`webhook.changed`)

6.11 Flow: Rahul books a big order → pending approval → Priya decides → Rahul sees why

Seed: tenant threshold ₹10,00,000; each shop has a credit limit and an opening balance. Figure 11 draws the same steps as a sequence.

#	Who · screen	Click or trigger	Endpoint	Server-side effect	What each side sees next
1	Rahul · Sale › review order	Adds lines; the **Credit check** card re-reads on every total change	`GET /v1/visits/:id/credit?amountPaise=`	read: exposure, projected exposure, `requiresApproval`, `reasons` (e.g. ORDER_THRESHOLD)	Card: "Needs approval — above approval threshold"; button label switches to **Send for approval**

#	Who · screen	Click or trigger	Endpoint	Server-side effect	What each side sees next
2	Rahul	**Send for approval**	`POST /v1/orders` + key	tx: Visit lock, proof check, pricing, **Shop lock + rule** → `PENDING_APPROVAL`, reasons and position frozen; outbox `order.created` + `order.pending_approval`	Rahul: "Sent for approval" screen; sync(`order.created`)
3	Rahul · visit workspace	Back to visit	`GET /v1/visits/:id` every **10 s** (foreground only) while any order on the visit or the shop's recent orders is PENDING_APPROVAL	read	Order row with amber note "Waiting for your manager" + the reasons sentence
4	Priya · Home / Orders	(polling) Home 10 s, Orders list and record 15 s, activity 10–20 s	`GET /v1/admin/home`, `/admin/orders`, `/admin/activity`	read: attention `ORDER_PENDING` "ORD-… awaiting approval · Sharma Jewellers · Rahul Verma · above approval threshold"; metric "pending approvals"; feed "ORD-… at Sharma Jewellers needs approval — above approval threshold"	Priya sees it within ~10 s of step 2
5a	Priya · order record or list preview	Reads "Credit check at booking" (frozen position) → **Approve order**	`POST /v1/admin/orders/:id/approve`	Shop lock, guarded update PENDING_APPROVAL → APPROVED (now outstanding), AuditLog, `order.approved`	Priya: sync(`order.decided`), button becomes status chip
5b	Priya	**Reject** → reason "Over limit — collect ₹2 L first" → **Reject order**	`POST /v1/admin/orders/:id/reject`	guarded update → REJECTED with `rejectionReason`, `decidedBy`, `decidedAt`; `order.rejected`	Priya: sync(`order.decided`)
6	Rahul · visit workspace	(next 10 s poll, or on focus)	`GET /v1/visits/:id`	read: `orders[].status/decidedBy/decidedAt/rejectionReason` (pinned by test P1)	`useVisit` notices waiting → decided and runs sync(`order.decided`); note reads "Rejected by Priya Mehta · 3:42 pm, 21 Oct — Reason: …" or "Approved by Priya Mehta"; polling stops

6.12 Flow: sell from memo with preview → approval → reject returns the pieces to the memo

#	Who · screen	Click or trigger	Endpoint	Server-side effect	What each side sees next
1	Rahul · visit workspace › Samples at this shop	Memo **Sell** → ticks AJ-S-000123 and AJ-S-000124	`GET /v1/memos/:id/convert-preview?pieceIds=…&visitId=`	read: one line per piece at today's rate next to its memo value at handover; subtotal, GST, total; `credit` with these pieces already out of memo exposure	Preview card; button **Book order for 2 pieces** or **Send 2 pieces for approval**
2	Rahul	Taps the button	`POST /v1/memos/:id/convert` + key	tx: Memo → Visit → Pieces locks; custody SHOP → SOLD (CONVERTED_TO_SALE); items CONVERTED; Shop lock + rule; order with `sourceMemoId` (e.g. PENDING_APPROVAL, CREDIT_LIMIT); memo status recomputed; outbox `sample.custody_changed`, `order.created`, `order.pending_approval`, `memo.converted`	Drawer: "Sent to your manager for approval"; sync(`memo.converted`)

#	Who · screen	Click or trigger	Endpoint	Server-side effect	What each side sees next
3	Priya · order record	**Reject**	`GET /v1/admin/memos/:id` (dialog only)	read	Dialog warns "Rejecting returns 2 pieces to MEMO-2026-000NN at Sharma Jewellers — they go back on memo, not to the vault."
4	Priya	Reason → **Reject order**	`POST /v1/admin/orders/:id/reject`	tx: `lockConversion` (Memo → sold Pieces) → Shop → guarded update REJECTED → **un-sell**: custody SOLD → SHOP (`SALE_REJECTED`), items CONVERTED → OUT, memo CONVERTED_TO_ORDER → ACTIVE, audit `memo.sale_rejected`; outbox `sample.custody_changed` + `order.rejected` with `unsold`; deferred sale guards pass at COMMIT	Priya: sync(`order.decided`, `order.unsold`): memo, samples, shop exposure and credit refresh; feed "Priya Mehta rejected ORD-… — 2 samples back on MEMO-…"
5	Rahul · visit workspace	(10 s poll)	`GET /v1/visits/:id`	read	Note: "Rejected by Priya Mehta … Reason: … The pieces went back on the memo at the shop." `useVisit` runs sync(`order.decided`, `order.unsold`); the two pieces are listed under the memo again (Return or Sell)

Cancel is refused for a conversion (`409 ORDER_NOT_CANCELLABLE`) because only a rejection can un-sell. Figure 12 is the sequence; Figure 5 shows the new SOLD → SHOP edge.

6.13 Flow: one event reaches every activity feed

#	Who · screen	Click or trigger	Endpoint	Server-side effect	What each side sees next
1	Any write (e.g. step 2 of §6.12)	—	the write's own route	`OutboxService.emit()` in the write transaction: `eventRefs(payload)` → `shopId, agentId, visitId, orderId, memoId` (or `sourceMemoId`), `pieceIds[]` stored on the row	Nothing yet; the row exists only if the write commits
2	Writer's browser	mutation succeeds	—	—	sync(change) invalidates the `["activity"]` prefix → every mounted `ActivityFeed` refetches its first page
3	Other admins	(polling)	`GET /v1/admin/activity?<ref>=…&limit=`	`ActivityService.page()`: indexed filter, keyset (createdAt, id), `project()` resolves ref labels in grouped queries	Home feed within 10 s; record Activity tabs within 20 s
4	Where the convert shows	—	`?shopId` · `?agentId` · `?visitId` · `?orderId` · `?memoId` · `?pieceId`	same `evt_…` ids and the same sentences everywhere: "Rahul Verma converted 2 samples on MEMO-… at Sharma Jewellers into ORD-…"	Home; Shop, Agent, Visit, Order and Memo records; each Sample record; record previews (compact)
5	Webhook consumer	(post-commit kick)	signed POST	same row, same `evt_…` id as the activity item	Figure 9

7 Flowcharts

Rendered from the code paths above. State names are the enum values stored in Postgres.

7.1 Request lifecycle

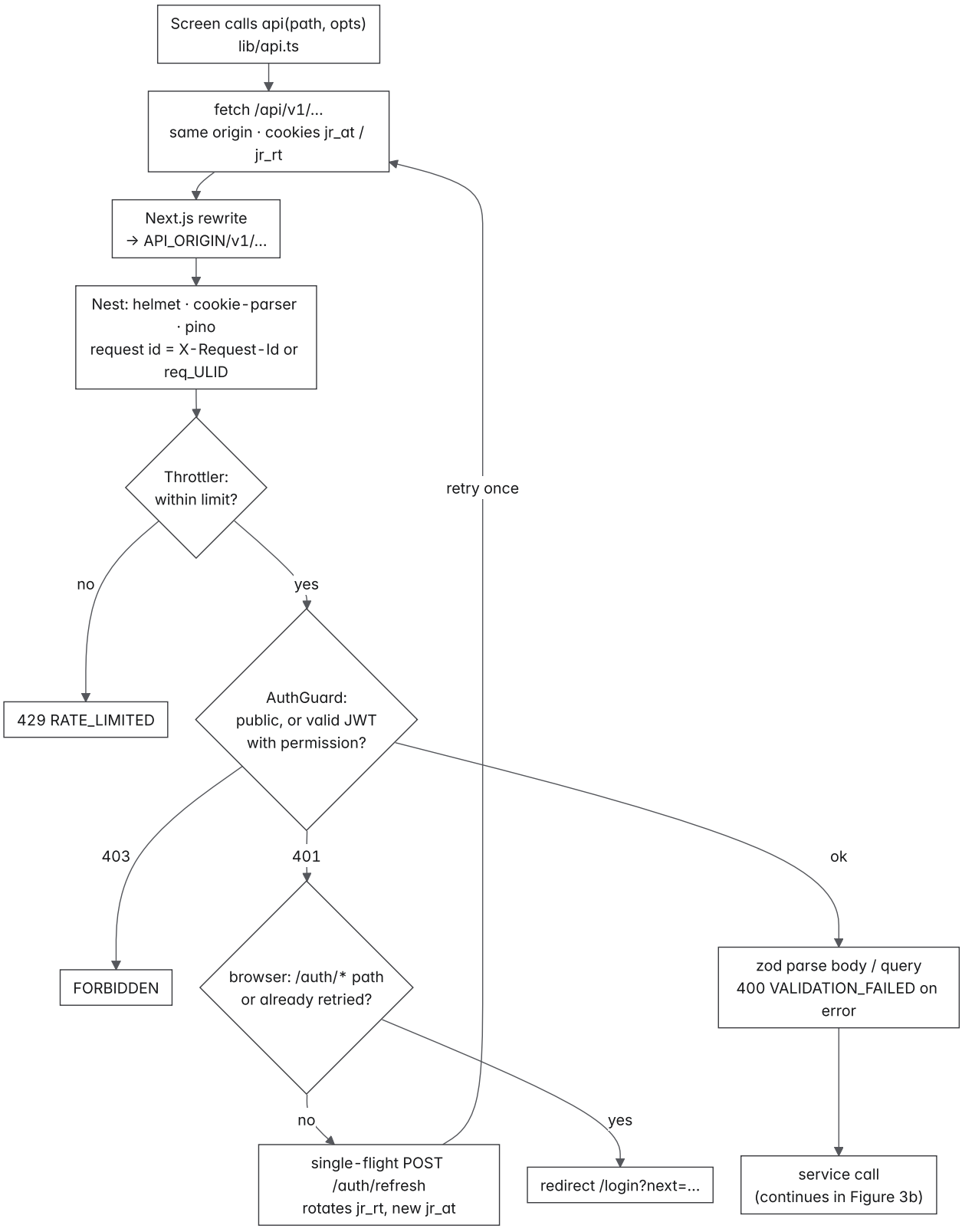


Figure 3a — Request lifecycle, part 1: transport, auth and validation. Every call, read or write, passes these steps. A 401 is refreshed once in the browser and replayed transparently.

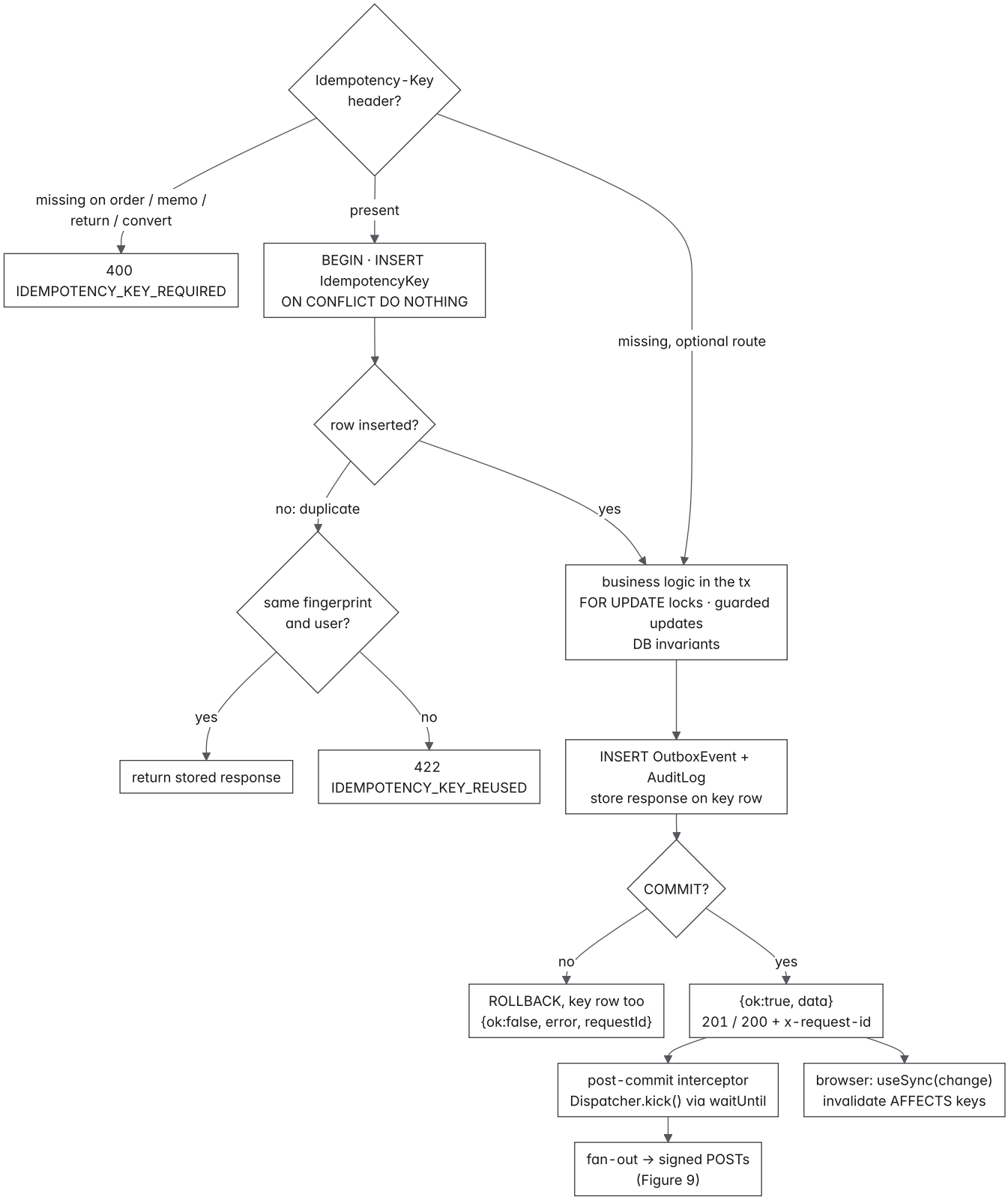


Figure 3b — Request lifecycle, part 2: idempotency, transaction, outbox, response, post-commit dispatch (mutating calls). Visit transitions (check-in, proof, complete) take the idempotency branch when the client sends a key; otherwise they run in a plain transaction.

7.2 Field visit flow

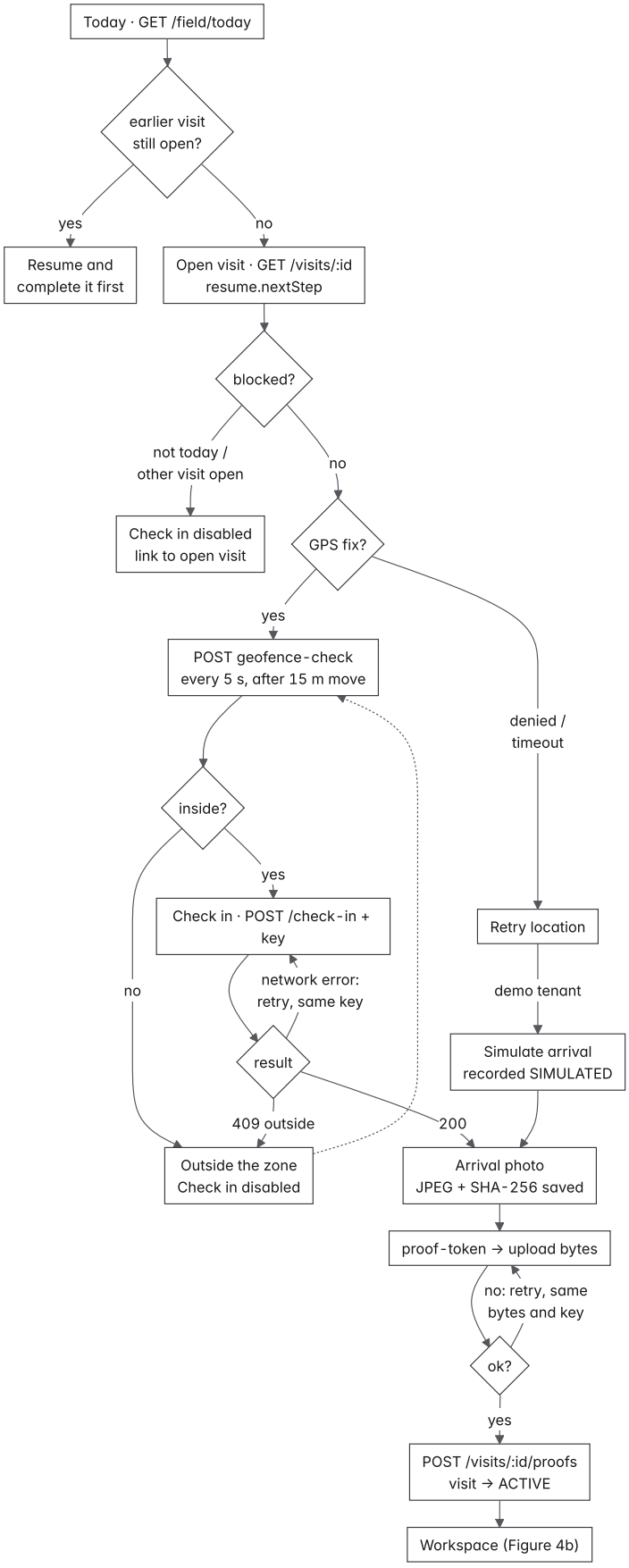


Figure 4a — Field visit flow, part 1: Today → geofence → check-in → arrival photo. Decision points for an earlier open visit, GPS denied, outside the fence, a network failure on check-in and upload failure. Every stage is re-derived from `GET /visits/:id`, so a reload at any point lands on the same step.

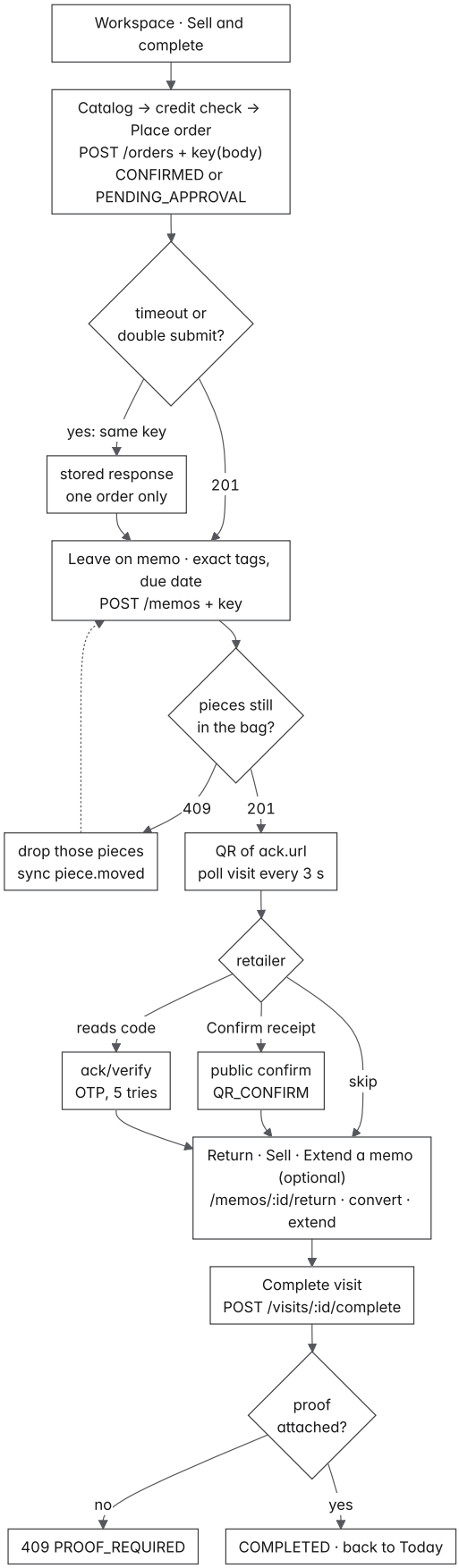


Figure 4b — Field visit flow, part 2: catalog → order → memo → retailer acknowledgement → return → complete. Shown in the usual order; in the app, order, memo, return and note are independent actions on the workspace and can be repeated or skipped. Duplicate submits are absorbed by the persisted idempotency key. Drafts (cart, memo picks, notes, pending photo, keys) live in sessionStorage per visit and are cleared when the server confirms.

7.3 Custody state machine for a sample piece

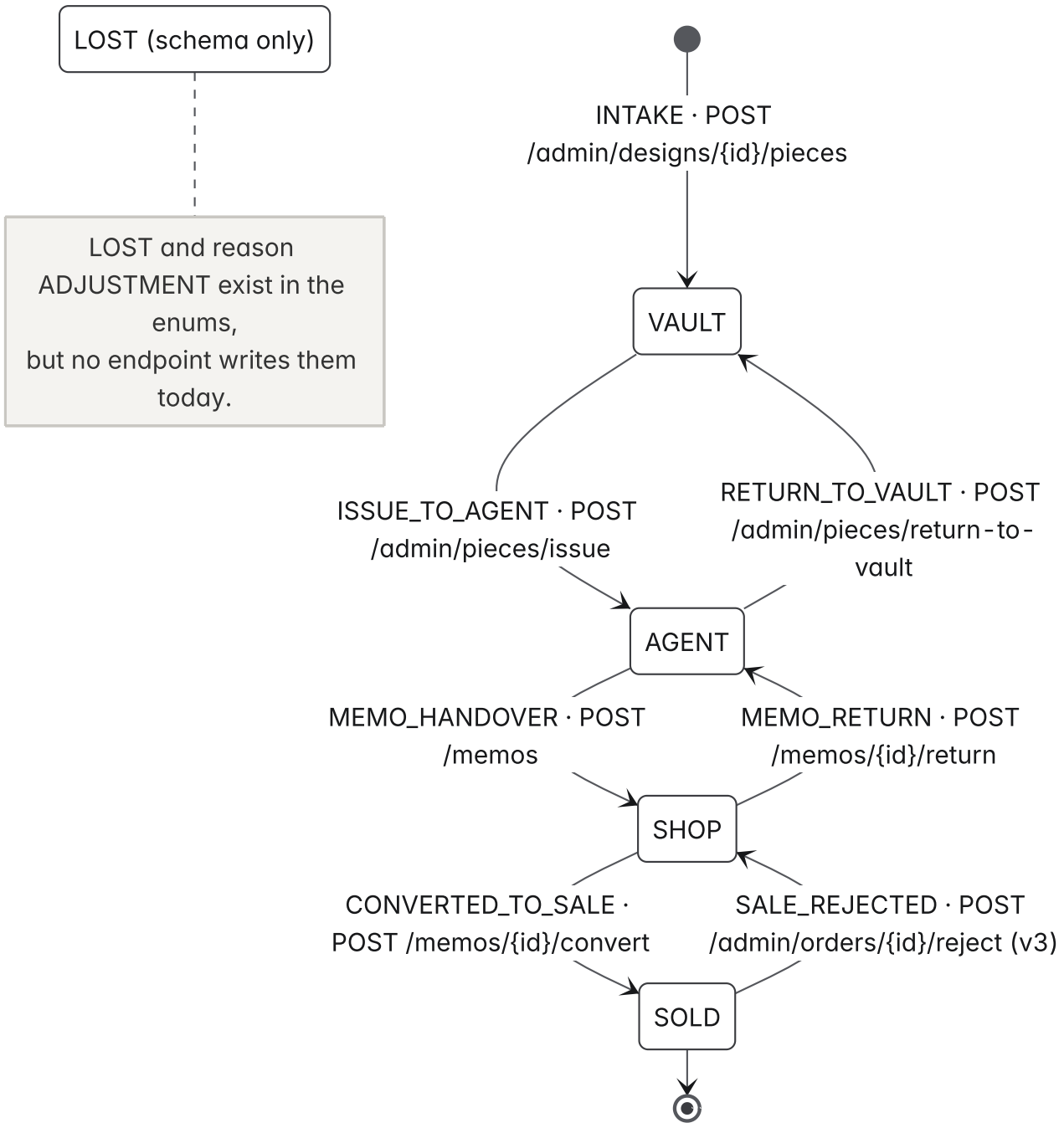


Figure 5 — Custody of one physical piece (for example `AJ-S-000123`). Every move goes through `CustodyService.move()`: lock the pieces `FOR UPDATE` in id order, verify the expected current custodian (else 409 with the specific code), update the piece, append a `CustodyEvent` with `seq+1`, emit one `sample.custody_changed`, and audit. MEMO_RETURN goes back to the memo's agent. SALE_REJECTED (v3) puts a piece sold from a memo back at the memo's shop when the manager rejects that sale; deferred triggers forbid a SOLD piece without a live order.

7.4 Memo state machine

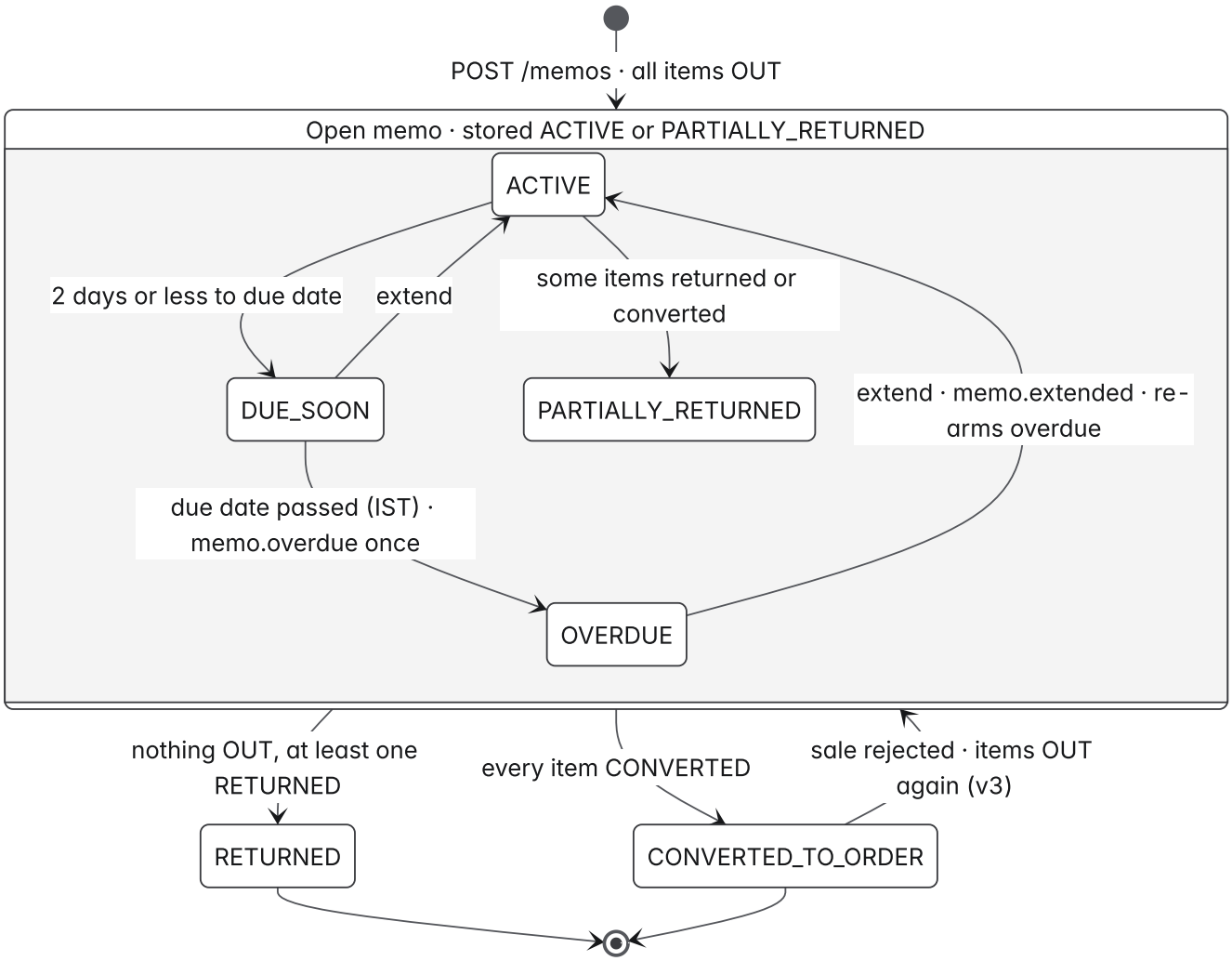


Figure 6 — Memo lifecycle. Stored `status` comes from `nextMemoStatus()`: all items OUT → ACTIVE; some OUT → PARTIALLY_RETURNED; none OUT → CONVERTED_TO_ORDER only if every item is CONVERTED, otherwise RETURNED (so a mix of returned and sold closes as RETURNED). DUE_SOON and OVERDUE are *display* states derived on read by `memoDisplayStatus()` and also apply to partially returned memos. "EXTENDED" is not a status: an extension writes a `MemoExtension` row and the view sets `extended: true`. Acknowledgement (`ackStatus` PENDING → ACKNOWLEDGED, event `memo.acknowledged`) is independent of these states. Since v3 a rejected conversion re-opens a memo: its items go CONVERTED → OUT and the status is recomputed, so a fully converted memo returns to ACTIVE (a partly converted one stays PARTIALLY_RETURNED or becomes ACTIVE).

7.5 Order state machine

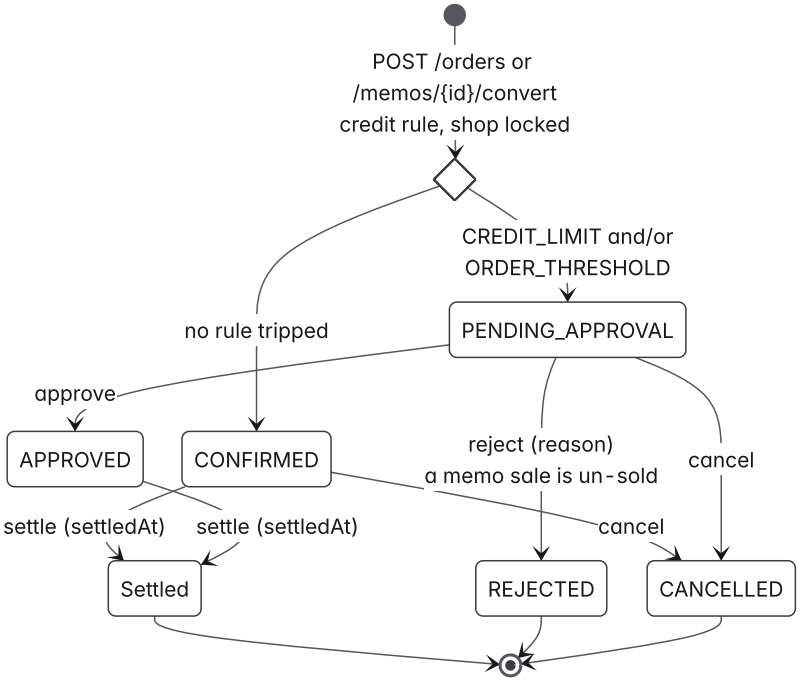


Figure 7 — Order lifecycle (v3). CONFIRMED and APPROVED count as outstanding until settled; "Settled" is `settledAt` on a CONFIRMED or APPROVED row, not a status. Decisions are one-shot (`updateMany … WHERE status='PENDING_APPROVAL'`; lost race → `ORDER_ALREADY_DECIDED`, other status → `ORDER_NOT_PENDING`). Cancel is refused once settled or for a memo conversion. Prices are frozen at creation. `DRAFT` is unused; `SUBMITTED` is forbidden by CHECK.

7.6 Visit state machine

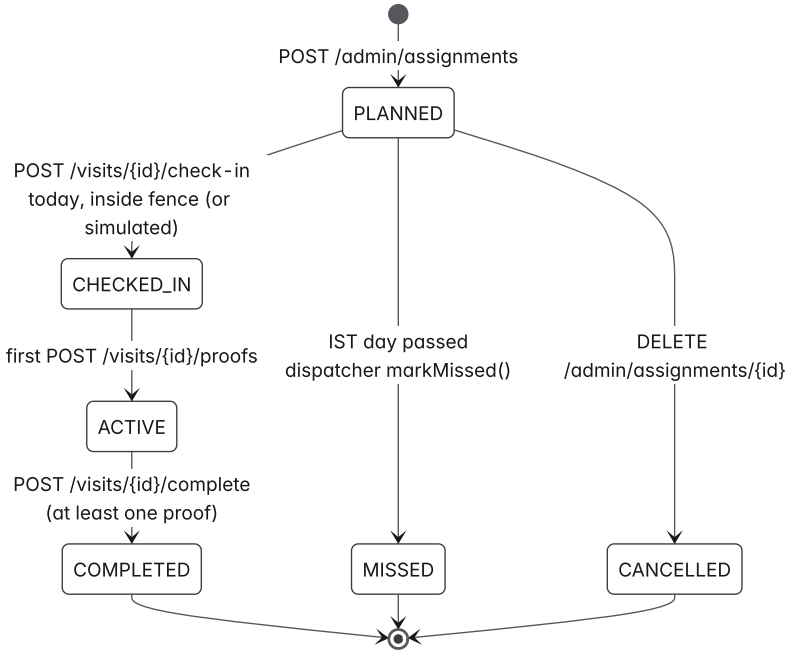


Figure 8 — Visit lifecycle. CHECKED_IN and ACTIVE are the "open" states: at most one per agent (partial unique index). Complete needs at least one proof, and the first proof moves the visit to ACTIVE, so COMPLETED is reached from ACTIVE in practice.

7.7 Webhook delivery pipeline

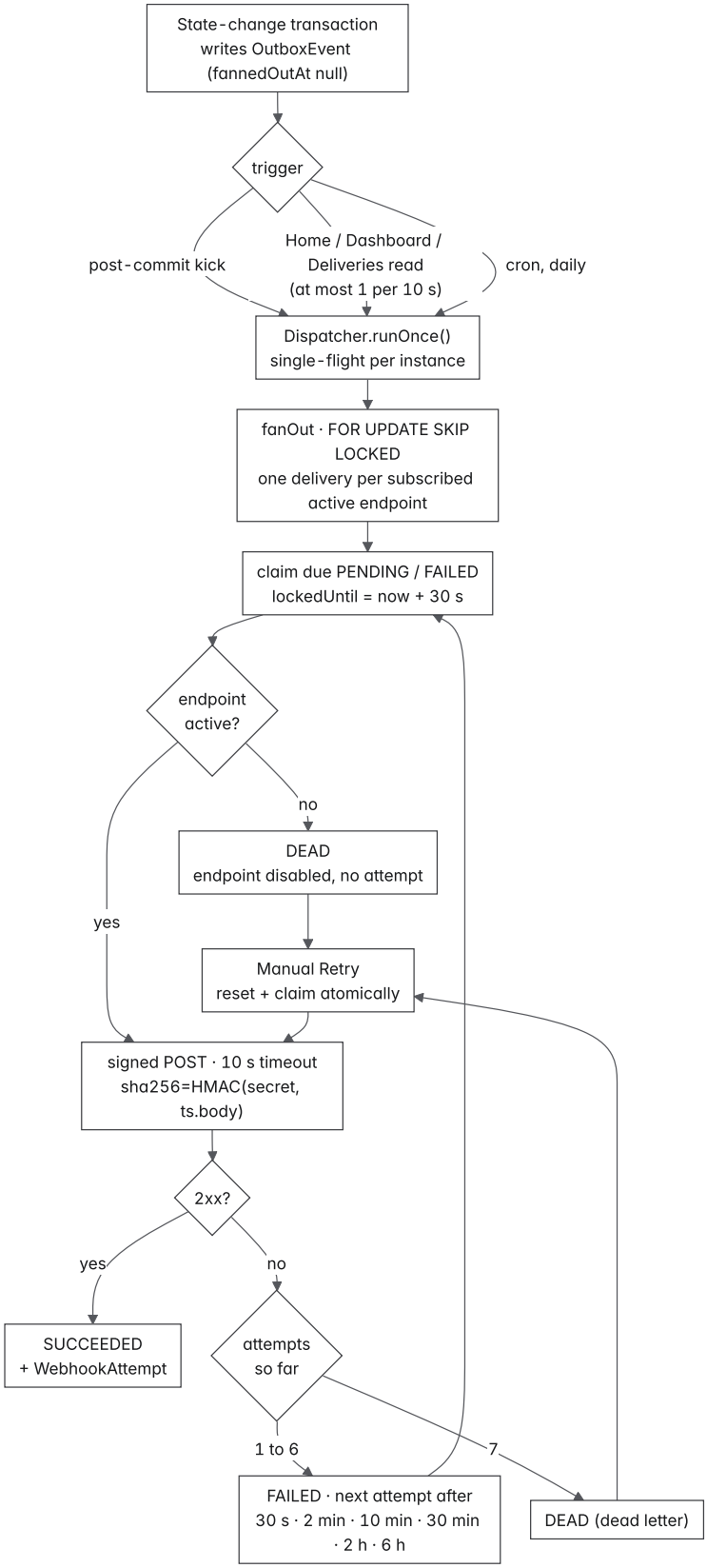


Figure 9 — Outbox → delivery → retries. One immediate attempt plus six retries after 30 s, 2 min, 10 min, 30 min, 2 h and 6 h; the 7th failure is DEAD. Each attempt records the headers actually sent (never the secret), status, latency and up to 2000 chars of the response. Back-off times are earliest-next-attempt; the attempt itself waits for the next trigger (§1.3).

7.8 Sequence: Rahul leaves AJ-S-000123 on memo at Sharma Jewellers

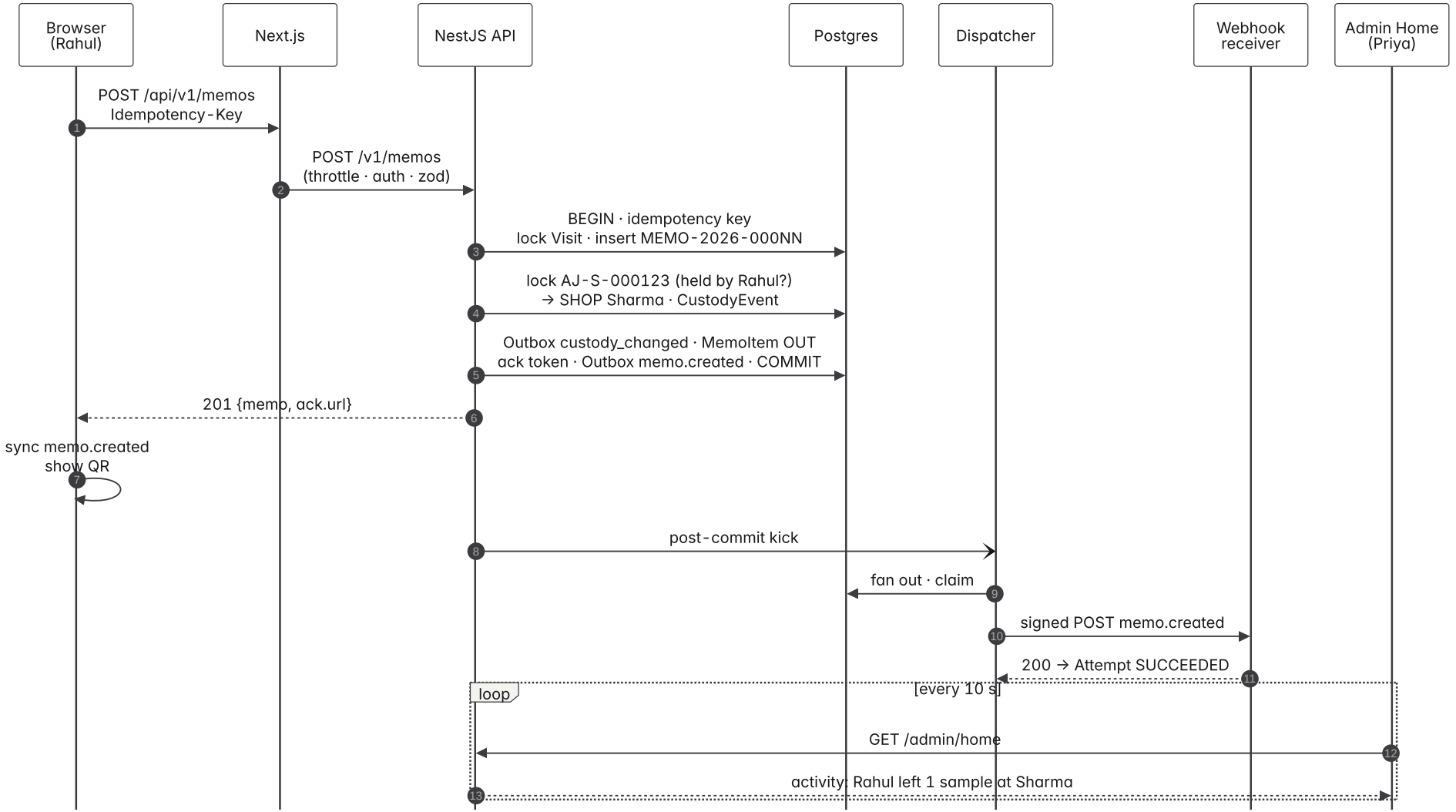


Figure 10 — One memo handover across all components. Steps 3–5 are one transaction: if any check fails (piece not in Rahul's bag, piece already on a memo, no gold rate), everything rolls back, including the idempotency row, and the browser shows the error with the draft intact.

7.9 Sequence: a big order waits for approval, Priya decides, Rahul sees why

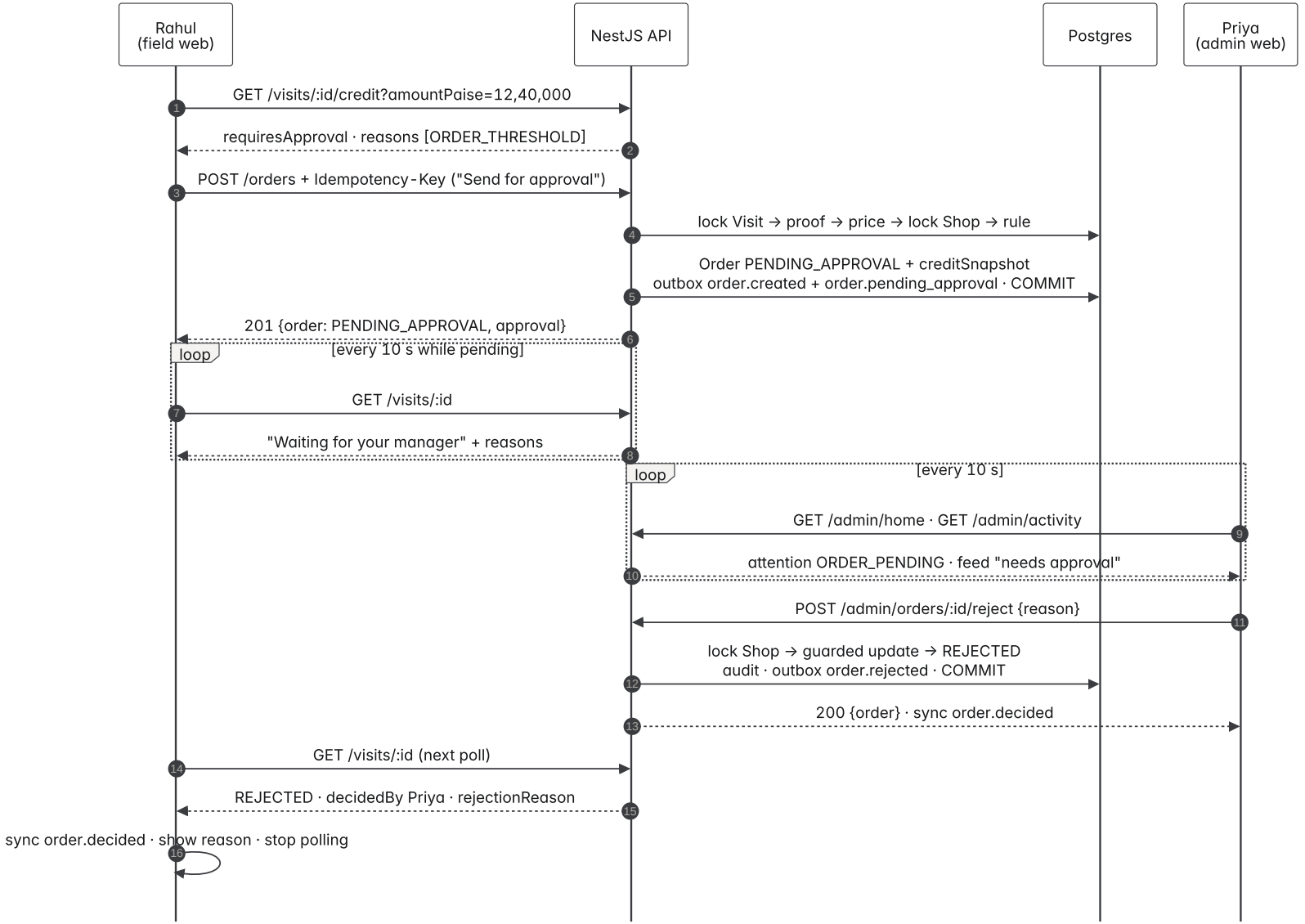


Figure 11 — Credit-aware order approval across both apps. The credit check before booking is advisory; the rule that counts runs again in step 4 with the shop row locked. An approve takes the same path as the reject (APPROVED, `order.approved`) and the order then counts as outstanding until settled.

7.10 Sequence: sell from memo with preview, then the sale is rejected and un-sold

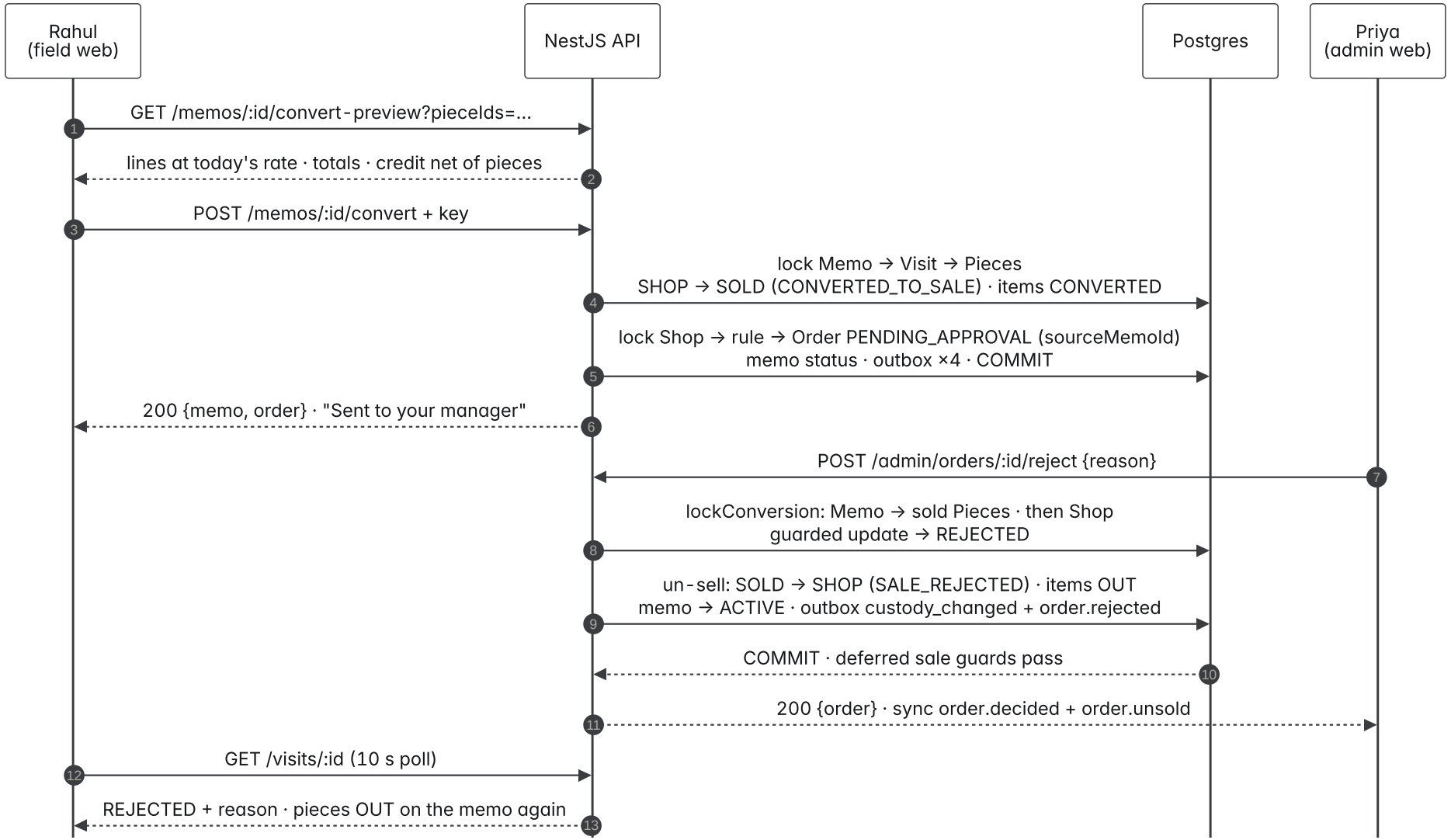


Figure 12 — Memo sale, rejection and un-sell. Both transactions take locks in the global order Memo → Visit → Pieces → Shop → Order, which is what removed the reject ‖ convert deadlock (test V13). If the un-sell were skipped, the deferred trigger `Order_dead_unsells_pieces` would refuse the COMMIT.

7.11 Activity propagation: one event, every feed

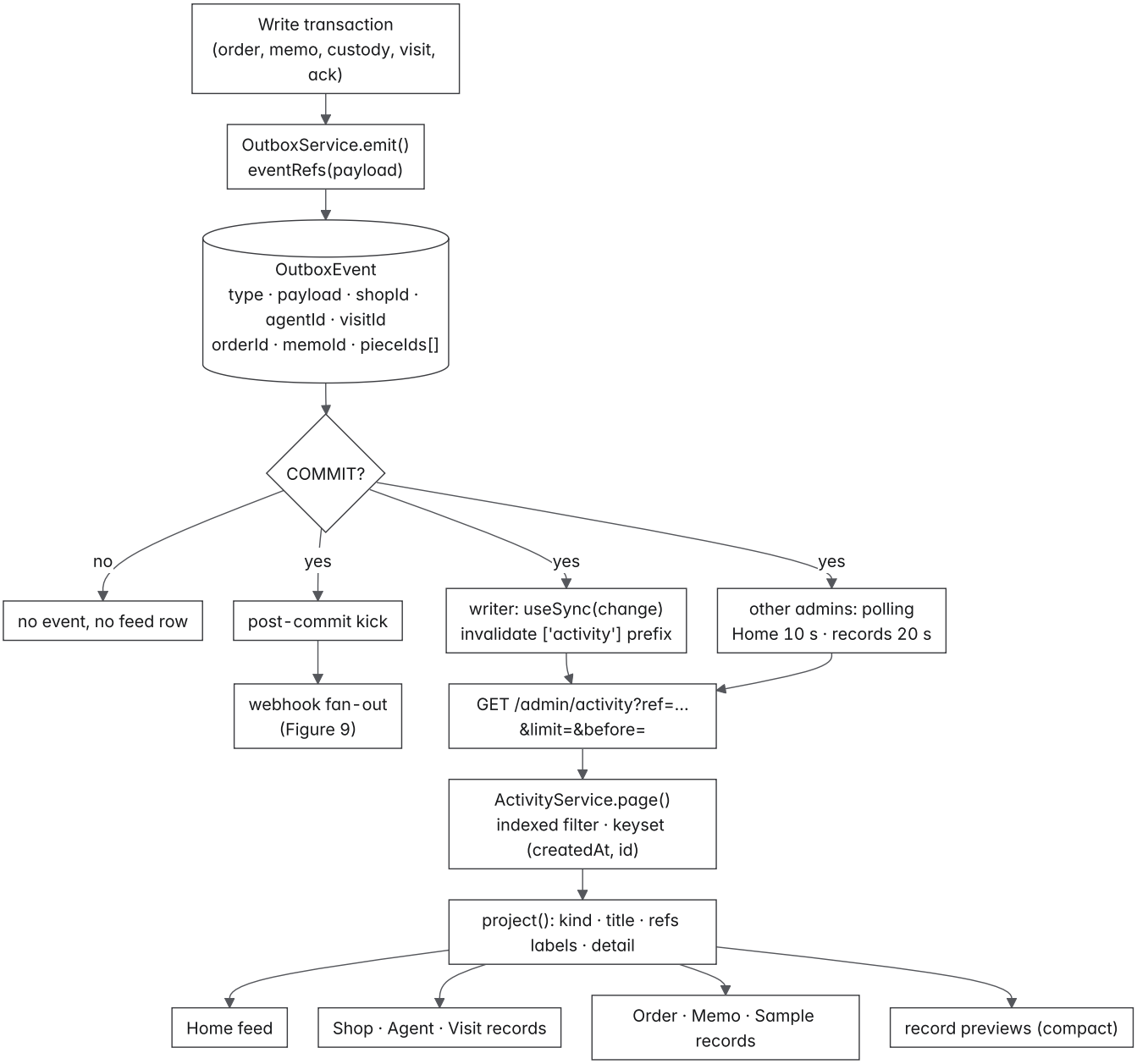


Figure 13 — One activity model. There is one writer (`OutboxService.emit`), one projection (`ActivityService`) and one renderer (`ActivityFeed`). The ActivityItem id is the webhook payload id, so a customer's log, the admin feed and the custody ledger name the same event.

8 Sync and consistency

8.1 The backend is the source of truth

The web holds no derived business state. After a write, it calls `useSync()(change)`, which invalidates every query key in `AFFECTS[change]` (`lib/sync.ts`). The server then answers with fresh numbers: bag values, memo exposure, shop status and attention items are all computed server-side. Keys are prefixes, so `["visit"]` refreshes every visit query. Local state is limited to drafts and UX: cart, memo picks, notes, the pending photo and idempotency keys in sessionStorage, plus the GPS fix. Pieces in a draft that the server no longer shows in the bag are dropped before submit. Cross-user freshness (agent writes, manager watches, or the reverse) comes from polling and refetch-on-focus:

Query (key)	Interval	Notes
Field Today `["today"]`	60 s + focus	Picks up new assignments and ack changes.
Field visit `["visit", id]`	focus; 10 s while an order is pending; 3 s on the ack screen	v3 (`POLL.pendingDecision`): polls while any order on the visit or the shop's recent orders is PENDING_APPROVAL, foreground only; a decision triggers sync(`order.decided`, + `order.unsold`). 3 s ack polling stops once ACKNOWLEDGED.
Credit check `["credit", …]` · sale preview `["convert-preview", …]`	on input change	Keyed by total or by piece selection; invalidated by every change that moves exposure (orders, custody, settle, cancel, settings, gold rate).
Retailer ack page (plain fetch)	5 s while pending + focus	`POLL.ackPage`; flips to "confirmed" when the agent enters the code.
Geofence `["geofence", id]`	5 s (foreground)	Plus 600 ms after moving ≥15 m; `staleTime: 0`.
Catalog `["catalog", …]`	stale 30 s	Invalidated by `gold_rate.set`, `design.saved` and custody changes.
Admin Home `["home", date]` · Home activity feed	10 s · 10 s	Home also kicks the dispatcher (≤1 per 10 s).
Record Activity tabs `["activity","feed",filter,limit]`	20 s	Infinite query with cursor pages; every write invalidates the `["activity"]` prefix.
Admin visits list and records; orders list and record; shop and agent records	15 s	`admin-visits`, `admin-visit`, `orders`, `order`, `shop`, `agent`.
Memos, pieces, piece and memo records	20 s	
Shops list · agents list and agent record	30 s · 15 s	Agent rows come from the single v2 `GET /admin/agents`.
Webhooks · deliveries (cursor pages) · delivery · received	15 s · 5 s · 5 s · 5 s	A delivery's state is visible within about 5 s of an attempt.
Everything else (TanStack default)	stale 10 s + focus	No retry on 4xx; up to 2 retries on 5xx or network errors.

8.2 Race and duplicate protections

Race	Protection (where)	Proven by
Double tap on a primary action	`useGuard()` ignores a second run before React re-renders; buttons disable while pending.	client code
Retry after timeout (order, memo, return)	Persisted `opKey` plus server idempotency in the same transaction; `clientOpId` unique backstop.	tests 3, F6, F7, F9
Concurrent double check-in	Visit row `FOR UPDATE`; partial unique index "one open visit per agent"; unique violation mapped to 409.	tests 1, 1b, F4
Same piece on two memos	Pieces locked `FOR UPDATE ORDER BY id` (deadlock-free), custodian verified; partial unique index on OUT memo items.	tests 4, 5, F8, F9
Issue a held piece, or two admins issue the same piece	`CustodyService.move` verifies `from = VAULT` under lock; gap-free `(pieceId, seq)` unique.	F10

Race	Protection (where)	Proven by
Approve and reject at once	`updateMany … WHERE status='PENDING_APPROVAL'`, one winner, one audit row; the loser's 409 names the winner. Batch approve reuses it per order.	tests 9, R4, V4
Two orders at one shop slip under the limit together	The credit rule runs with the shop row `FOR UPDATE`; the second order waits and sees the first (READ COMMITTED). Settle and cancel take the same lock.	V1, V2
Reject ‖ convert on the same memo (deadlock)	One global lock order Memo → Visit → Pieces → Shop → Order; `lockConversion` takes Memo and the sold pieces before the Shop.	V13 (6 rounds)
A rejected or cancelled sale leaves a piece SOLD	Un-sell in the same transaction; deferred triggers `SamplePiece_sold_has_live_order` / `Order_dead_unsells_pieces` refuse the COMMIT otherwise; cancel of a conversion is refused. Ops check `db:check-orphans` for old rows.	V10, V14
Gold rate changes mid-order	Rates read once per transaction; lines and snapshot use the same map; OrderItem is append-only.	tests 8, F11
Two dispatchers (instances or cron)	`FOR UPDATE SKIP LOCKED` on fan-out and claim; `lockedUntil` lease; unique (endpoint, event); single-flight per instance; manual retry resets and claims in one statement.	tests 7, F12
Overdue event spam	`updateMany … WHERE overdueNotifiedAt IS NULL` per memo; extension resets it.	F13
Return or sale in a later visit	Attributed to the caller's open visit at the memo's shop (custody event, outbox, converted order).	F14

8.3 Failure handling

Failure	What happens	Proven by
Bad or far GPS fix	Server-side distance check; accuracy slack capped at 50 m, so a wildly inaccurate fix cannot widen the fence. Missing accuracy earns no slack and is never VERIFIED (LOW_ACCURACY). The UI enables Check in only on a fresh server verdict.	test 2, F3
GPS denied, unavailable or timing out	`useFieldGeo` maps each case to specific copy with **Retry location**. A demo tenant offers **Simulate arrival**, recorded SIMULATED and flagged on Home.	F16 (seed demos)
Upload failure	Prepared bytes, checksum and key are persisted. Retry resumes from the failed step with the same bytes and key. The server dedupes on (visit, checksum), and direct uploads upsert by key.	client + unique index
API error mid-submit, retry	The error rolls back the key row, so the retry re-executes rather than replaying an error. A 5xx keeps the client key; a 4xx clears it.	F6
Refresh mid-visit	`resume` block from `GET /visits/:id`; drafts in sessionStorage; a pending photo comes back as "retry". A visit left open on an earlier day is surfaced on Today.	F5, F15; AUDIT #4–5
Session expiry	A 401 triggers single-flight refresh and one retry; the upload XHR refreshes via `/auth/me`; otherwise redirect with `?next=`.	client code
Tenant isolation	tenantId only from the JWT; every query filtered; foreign ids 404 on every v1 and v2 read and write; lists, search and home never contain them.	test 6, F1
Order submit times out	`sendConfirmed()` re-sends with the same key ("Checking") before telling the agent anything; a stored response confirms the one order. Still unknown → draft kept, **Retry** with the same key.	client code + test 3
Preview and write disagree	The credit check and the memo-sale preview are advisory; the convert and order create re-evaluate under their locks and return the authoritative `order.approval`, which the UI renders after booking.	P2–P4
Webhook endpoint down	FAILED with back-off → visible in Deliveries and as a Home attention item → manual retry after fixing the URL → SUCCEEDED.	test 7, F12

Test assets. `apps/api/test/api.test.ts` (12 cases: invariants 1–9 plus 1b, 1c proof-before-sale, 7b demo receiver), `failures.test.ts` (F1–F16), `records.test.ts` (R1–R4), `v3.test.ts` (V1–V14: rule boundaries, concurrent orders, convert net of pieces, status guards, PATCH keeps fields, refs on every event, activity filters and cursor, settings, legacy migration, un-sell, rollback, back-fill, lock order, orphans), `p7.test.ts` (P1–P4: decision fields, convert preview, attention timing) on real Postgres via `test/harness.ts`; `packages/shared pricing.test.ts` (4), `credit.test.ts` (6),

`format.test.ts` (4); `apps/api/scripts/e2e-smoke.ts` (31 checks over HTTP with sample `AJ-S-000123`, re-runnable); reviews and verification in `docs/brain/reviews/` and `docs/VERIFY-P7.md`. These were not run for this report.

9 Appendix

9.1 Environment variables

Variable	App	Purpose (validated in `apps/api/src/env.ts` with zod)
`API_ORIGIN`	web	Where `/api/*` is proxied (default `http://localhost:4000`).
`NODE_ENV` · `PORT`	api	development|test|production (production → secure cookies, https-only webhooks) · default 4000.
`DATABASE_URL` · `DATABASE_URL_DIRECT`	api	Pooled URL for the app · direct URL for `prisma migrate` (defaults to `DATABASE_URL`).
`JWT_ACCESS_SECRET`	api	HMAC key for access JWTs (≥16 chars).
`JWT_REFRESH_SECRET`	api	Required (≥16) but currently unused; refresh tokens are opaque and hashed.
`APP_ENCRYPTION_KEY`	api	32-byte base64 AES-256-GCM key for webhook secrets at rest.
`APP_SECRET`	api	HMAC key for retailer ack codes and their hashes.
`CRON_SECRET`	api	Guards `/v1/internal/dispatch` and `/v1/internal/seed` (header or Bearer).
`APP_PUBLIC_URL` · `API_PUBLIC_URL`	api	Web origin for ack links and QR codes and absolute proof URLs in events · API origin for demo receiver URLs.
`CORS_ORIGINS`	api	Comma-separated allowlist (fallback; the proxy makes calls same-origin).
`BLOB_READ_WRITE_TOKEN`	api	Optional. Enables Vercel Blob client uploads; otherwise direct mode.
`LOG_LEVEL`	api	pino level (default info).
`ENABLE_DEMO_RECEIVER`	api	Optional. `true` enables `/dev/webhook-receiver`; unset = on outside production, or in production only for demo-mode tenants; `false` = always 404.

9.2 Running locally

- One command: `./scripts/run-local.sh` (Node 22+, with Docker or a local Postgres 16). It writes `apps/api/.env` with fresh secrets, installs dependencies, builds shared and the API, runs `prisma migrate deploy`, starts the API on :4000, seeds via `POST /v1/internal/seed`, then starts the web on :3000.
- Manual: `pnpm install` → `docker compose up -d db` → `cp apps/api/.env.example apps/api/.env` (fill secrets) → `pnpm --filter @jewelroute/api db:migrate && pnpm --filter @jewelroute/api db:seed` → `pnpm --filter @jewelroute/api dev` and `pnpm --filter @jewelroute/web dev`.
- Demo logins: `rahul@aurum.test` / `Rahul@2026` (field), `admin@aurum.test` / `Aurum@2026` (manager). Swagger at `http://localhost:4000/docs`. Away from Delhi, use **Simulate arrival (demo)**.
- Tests: `pnpm --filter @jewelroute/api test` (needs a test database); e2e: `cd apps/api && BASE_URL=http://localhost:4000 CRON_SECRET=… pnpm e2e`.
- v3 ops scripts: `pnpm --filter @jewelroute/api db:backfill-activity-refs` (batched, idempotent; runs after migrate on Vercel and in `run-local.sh`) and `db:check-orphans` (read-only; exit 1 if a SOLD piece has no live order). Rollback SQL: `apps/api/prisma/rollback/20261021000300_v3_down.sql`.
- This report: `node docs/report/build.mjs` writes `docs/report/out/JewelRoute-FE-BE-Integration-Report.pdf` (see the header of that file for the Playwright and Mermaid paths).

9.3 Glossary

Memo	Consignment note: physical samples left with a retailer until a due date, then returned or bought. Numbered `MEMO-YYYY-NNNNN`; acknowledged by the retailer through QR or code.

Custody / custodian	Who physically holds a sample right now: the VAULT, an AGENT (their "bag"), a SHOP (on memo), SOLD or LOST. Changes only through the append-only `CustodyEvent` ledger.
Sample piece · tag code	One physical copy of a design, identified by a tag such as `AJ-S-000123`, with its own gross and net weight.
Design · SKU	A catalog item (for example a 22K bangle) that defines purity, standard weights and pricing components.
Purity	Gold fineness in karats: K24 (≈99.9%), K22 (≈91.6%), K18 (75%), K14 (≈58.5%). Each purity has its own daily rate per gram.
Paise / mg	Integer units: 1 rupee = 100 paise; 1 gram = 1000 milligrams. Price = net mg × rate (paise per g) ÷ 1000 + making + stones + other, then 3% GST on the order subtotal.
Making charge	Labour charge, either per gram of net weight (paise per g) or a percentage of gold value (basis points).
Gross / net weight	Total weight vs. gold-only weight (excludes stones); pricing uses net.
Geofence · verification	Circle of `geofenceRadiusM` around a shop; check-in records VERIFIED, LOW_ACCURACY or SIMULATED.
Proof	The arrival photo attached to a visit, stored with its SHA-256 checksum and capture position.
IST business date	The calendar day in Asia/Kolkata used for "today", due dates and missed visits.
Outbox · fan-out · dead letter	Event rows written with each change · creating one delivery per subscribed endpoint · a delivery that failed 7 times (DEAD), recoverable by manual retry.
Credit limit · outstanding · exposure	The shop's limit (0 = none) · opening balance + confirmed/approved orders not yet settled · outstanding + value of samples on memo at the shop. Projected exposure adds a prospective order.
Approval threshold · pending approval	Tenant setting: orders of at least this total need a manager · the order status when the credit limit or the threshold rule trips.
Settle · un-sell	Mark an order paid, removing it from outstanding · on rejecting a memo sale, return its pieces from SOLD to the memo's shop (custody reason SALE_REJECTED).
Activity refs	Indexed columns on each outbox event naming the shop, agent, visit, order, memo and pieces it is about; every feed filters on them.
Idempotency key	A client-chosen id per user action; repeating the request with it returns the original result instead of acting twice.